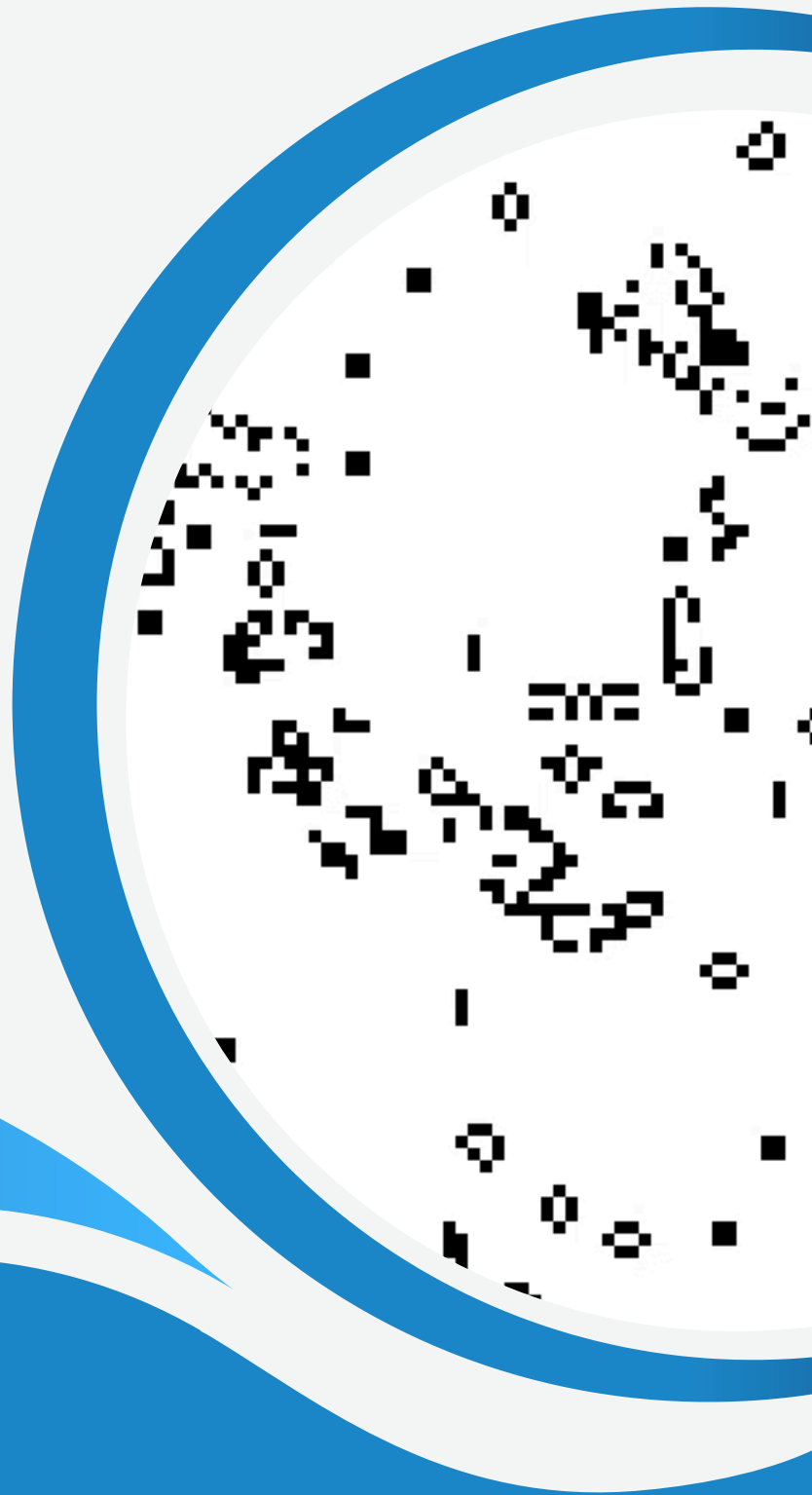


DOCUMENTATION TECHNIQUE ET UTILISATEUR

SYSTEME :
JEUX DE LA VIE

présenté par :
MADANI MUSTAPHA
MAUMY ALEXANDRE



SOMMAIRE

PARTIE TECHNIQUE

Structure du code

- Présentation des diagrammes :
 - Diagramme de cas d'utilisation
 - Diagramme de classe
 - Diagramme de séquence
 - Diagramme d'activité

1

Implémentation du code

- Le fichier Makefile
- Détail de fonctionnalités de toutes le classes

2

Tests Unitaires

- Présentation des tests unitaires et leurs buts

3

PARTIE UTILISATEUR

Présentation du jeu de la vie

- Jeu de la vie
- Explication des règles du Jeu de la vie

4

Obtention du code

- Etapes et prérequis pour pouvoir lancer le jeu
- Comment obtenir le jeu (le code) et les versions

5

Lancement du jeux

- Comment lancer le jeu
- Les résultats attendus

6

Annexe

7

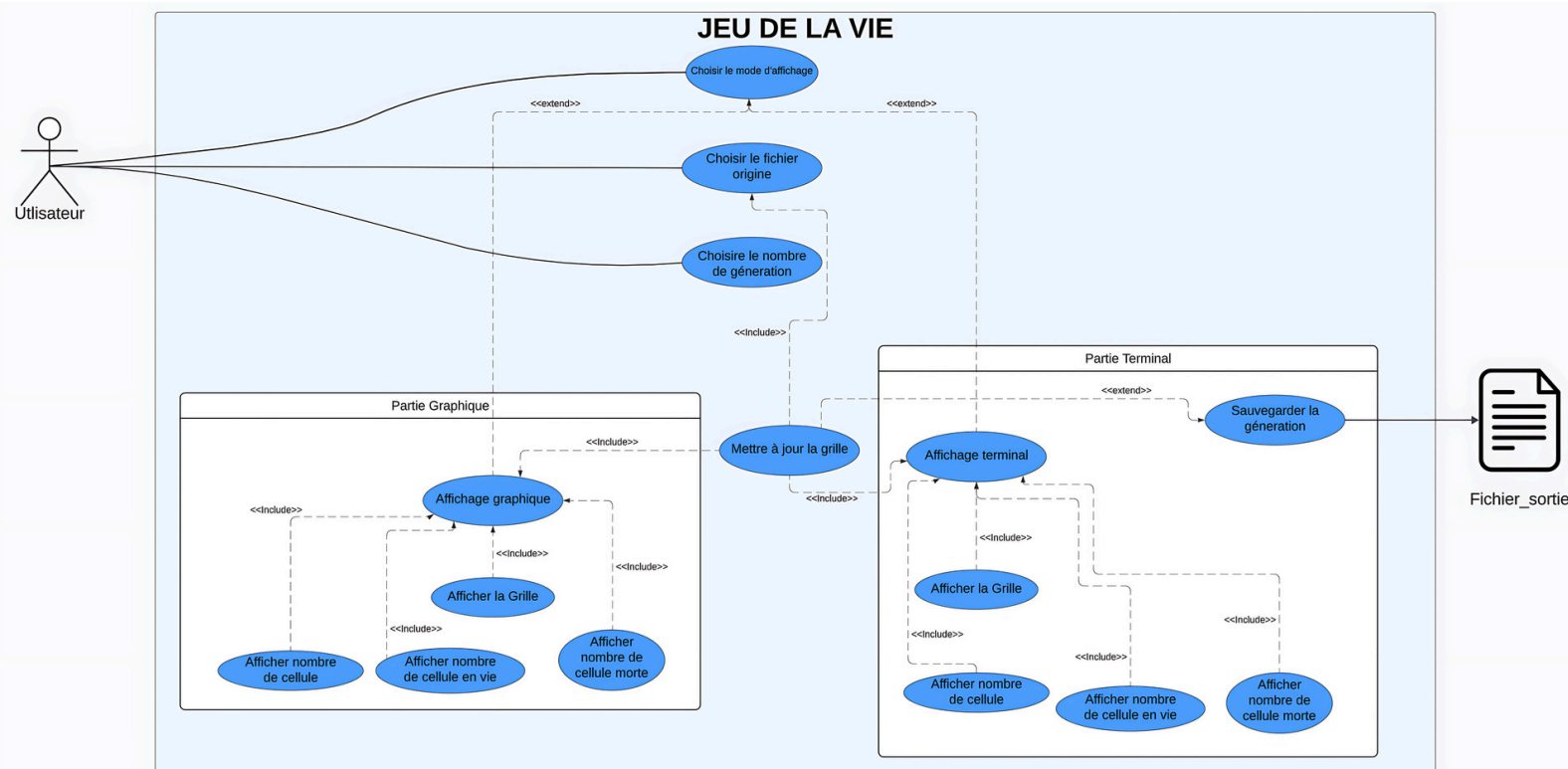
PARTIE TECHNIQUE

STRUCTURE DU CODE

PRÉSENTATION DES DIAGRAMMES

Diagramme de cas d'utilisation :

Un diagramme cas d'utilisation est une liste d'étapes qui définissent les interactions entre un acteur (un humain qui interagit avec le système ou un système externe) et le système lui-même. Les diagrammes de cas d'utilisation décrivent les spécifications d'un cas d'utilisation et modélisent les unités fonctionnelles d'un système. Ces diagrammes aident les équipes de développeurs à comprendre les besoins de leur système, notamment le rôle des interactions humaines et les différences entre plusieurs cas d'utilisation. Un diagramme d'utilisation peut illustrer tous les cas d'utilisation du système ou seulement un groupe de cas d'utilisation ayant des fonctionnalités similaires.



Identification des besoins :

A travers ce diagramme, nous avons donc identifié les besoins suivants :

- Choisir un mode de jeu ;
- Pouvoir choisir le fichier source pour générer la grille ;
- Pouvoir choisir le nombre de génération ;
- Afficher le jeu dans une version console ;
- Afficher le jeu dans une version graphique ;
- Sauvegarder les générations dans un dossier.

Composants de notre diagramme :

Nous avons tout d'abord les **limites** du système qui permettent de définir le champs d'action de ce dernier :



Composants de notre diagramme :

Ensuite, nous avons l'**acteur** de notre système qui sera notre utilisateur :

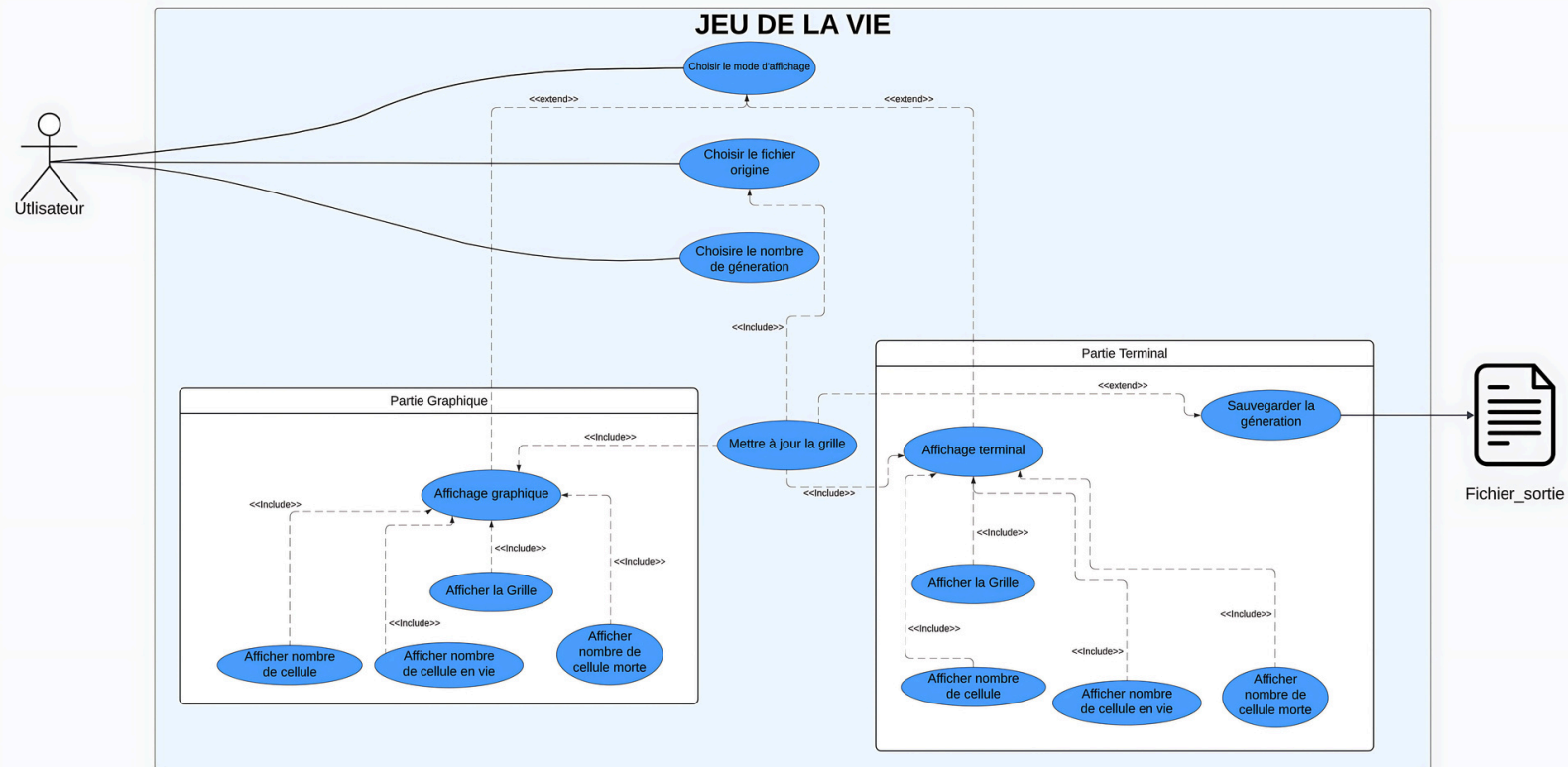
Cet utilisateur pourra :

- Choisir le mode de jeu auquel il souhaite jouer ;
- Choisir le fichier source ;
- Choisir le nombre de génération.



Composants de notre diagramme :

Ensuite, il y a le **système** en lui-même :



Composants de notre diagramme :

Dans un premier temps, nous avons la fonctionnalité « Choisir le mode d'affichage » : cette dernière permet comme son nom l'indique de choisir la manière dont le jeu va être affiché. Il existe deux modes, soit graphique, soit console.

Ensuite, il y a la fonctionnalité « Choisir le fichier d'origine » : cela permet d'entrer le premier fichier qui va permettre de générer la grille de départ pour notre automate cellulaire.

Puis, il y a celle de « Choisir le nombre de génération » : cette dernière permet de choisir un nombre défini d'itération que le programme réalisera. Par exemple, si l'utilisateur entre 5 à cette demande alors il y aura 5 générations de cellules.

Enfin, il y a la fonctionnalités « Mettre à jour la grille » qui permet comme son nom l'indique de mettre à jour la grille.

Composants de notre diagramme :

Ensuite, on peut voir deux autres blocs dans notre système :

- « Partie graphique »
- « Partie Terminal »

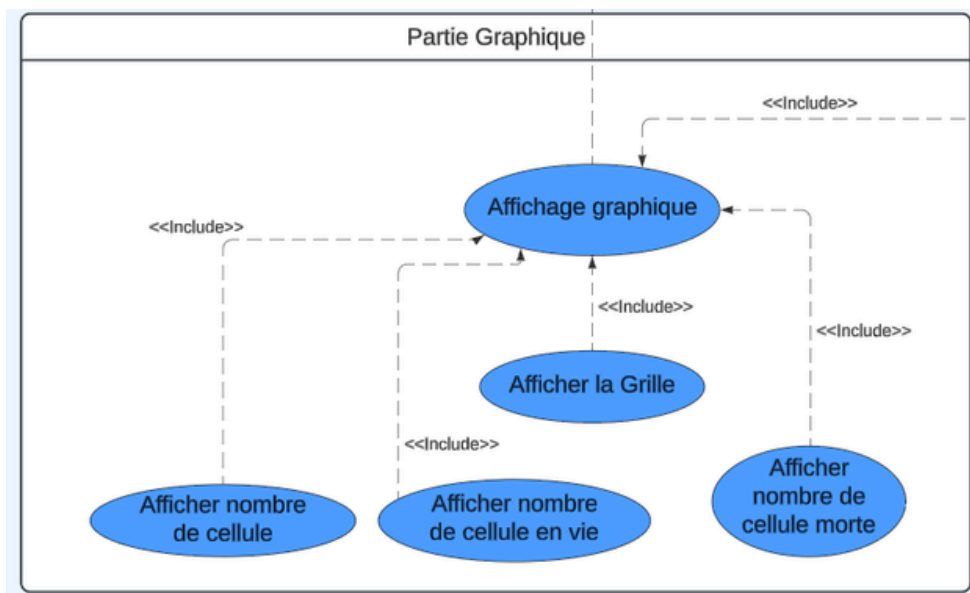
Composants de notre diagramme :

La section « Partie graphique » gère l'affichage de informations dans une interface graphique. En voici les fonctionnalités :

- « Affichage graphique » : cette fonctionnalité regroupe les 4 fonctionnalités ci-après.
- « Afficher la grille » : la grille est affichée sur l'interface graphique
- « Afficher le nombre de cellule » : affiche le nombre totale de cellule présente dans la grille
- « Afficher le nombre de cellule en vie » : affiche le nombre de cellules en vie présentes sur la grille
- « Afficher le nombre de cellule mortes » : affiche le nombre de cellules mortes encore présentes sur la grille.

On peut voir différents liens :

- Un lien « extend » entre « Affichage graphique » et « Choisir le mode d'affichage » : en effet, n'est pas nécessairement graphique.
- Un lien « include » entre « Affichage graphique » et « Mettre à jour la grille » : en effet, la mise à jour de la grille est obligatoire sinon nous observerons aucun évolution.
- Un lien « include » entre « Affichage graphique » et « Afficher le nombre de cellule » : cela est obligatoire pour suivre le nombre de cellule présente sur la grille afin de suivre l'évolution.
- Un lien « include » entre « Affichage graphique » et « Afficher le nombre de cellule vivante » : cela est aussi obligatoire pour avoir une vision plus claire des cellules vivantes présentes sur la grille.
- Un lien « include » entre « Affichage graphique » et « Afficher le nombre de cellule morte » : cela est aussi obligatoire pour avoir une vision plus claire des cellules mortes présentes sur la grille.
- Un lien « include » entre « Affichage graphique » et « Afficher la grille » : ce lien est le plus important car sans affichage on ne peut rien observer au niveau de l'évolution.



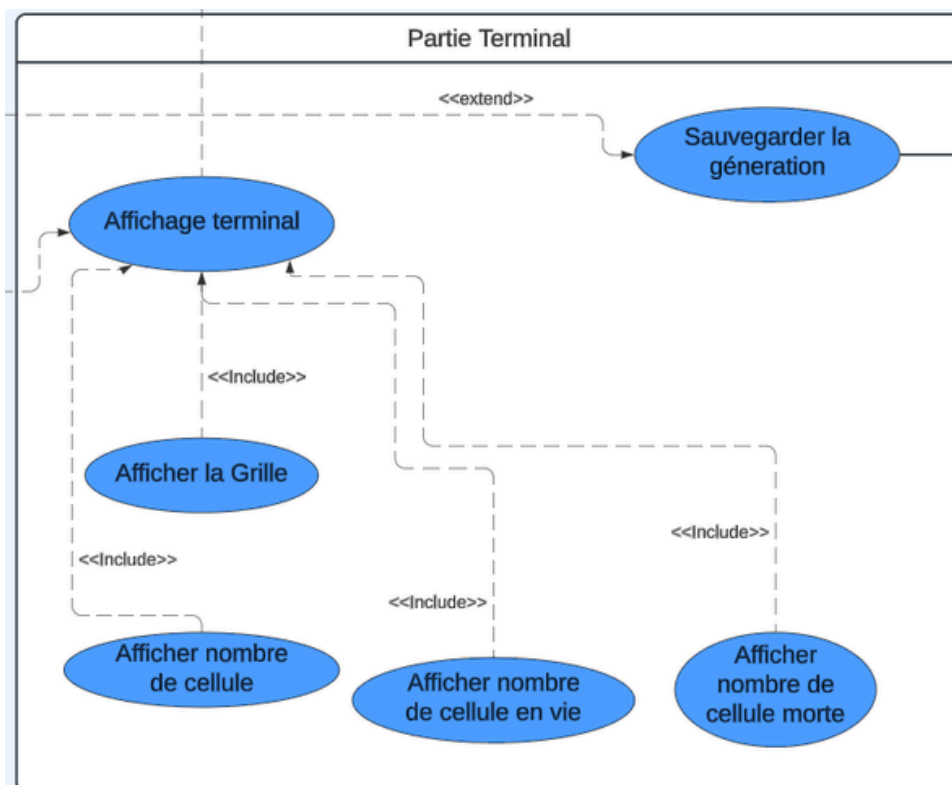
Composants de notre diagramme :

Ensuite, nous avons la section « Partie terminal » qui gère l'affichage de la grille dans la console.

- « Affichage terminal » : cette fonctionnalité regroupe les 5 fonctionnalités ci-après.
- « Afficher la grille » : la grille est affichée sur le terminal
- « Afficher le nombre de cellule » : affiche le nombre totale de cellule prent dans la grille
- « Afficher le nombre de cellule en vie » : affiche le nombre de cellules en vie présentes sur la grille
- « Afficher le nombre de cellule mortes » : affiche le nombre de cellules mortes encore présentes sur la grille
- « Sauvegarder les fichiers » : affiche le nombre de cellules mortes encore présentes sur la grille.

On peut voir différents liens :

- Un lien « extend » entre « Affichage terminal » et « Choisir le mode d'affichage » : en effet, n'est pas nécessairement dans la console.
- Un lien « include » entre « Affichage terminal » et « Mettre à jour la grille » : en effet, la mise à jour de la grille est obligatoire sinon nous observerons aucun évolution.
- Un lien « include » entre « Affichage graphique » et « Afficher le nombre de cellule » : cela est obligatoire pour suivre le nombre de cellule présente sur la grille afin de suivre l'évolution.
- Un lien « include » entre « Affichage graphique » et « Afficher le nombre de cellule vivante » : cela est aussi obligatoire pour avoir une vision plus claire des cellules vivantes présentes sur la grille.
- Un lien « include » entre « Affichage graphique » et « Afficher le nombre de cellule morte » : cela est aussi obligatoire pour avoir une vision plus claire des cellules mortes présentes sur la grille.
- Un lien « include » entre « Affichage graphique » et « Afficher la grille » : ce lien est le plus important car sans affichage on ne peut rien observer au niveau de l'évolution.
- Un lien « extend » entre « Mettre à jour la grille » et « Sauvegarder la génération » : en effet, la sauvegarde n'est qu'une action facultative, nous ne sommes pas obligés de les sauvegarder.



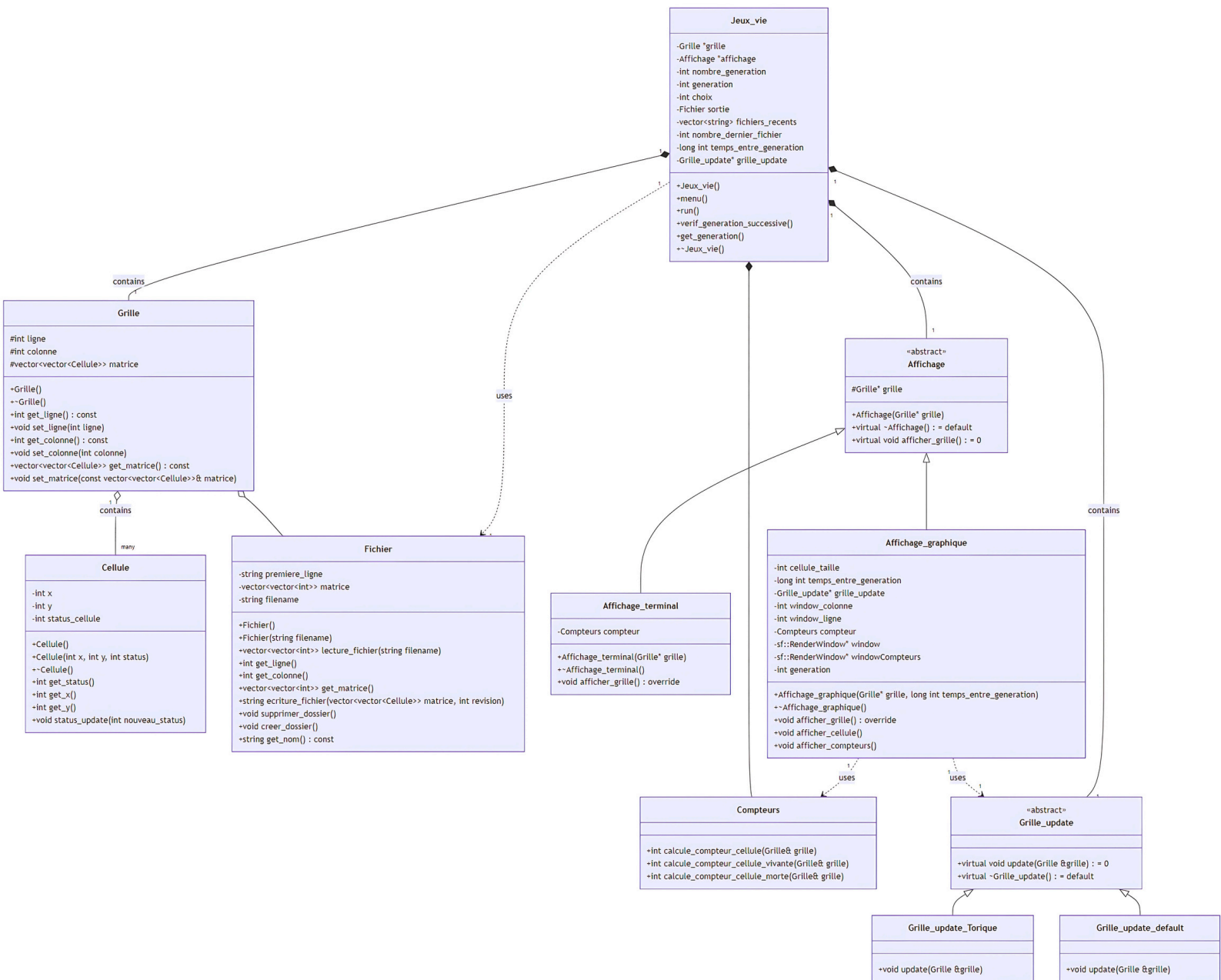
Composants de notre diagramme :

Enfin, nous avons l'acteur extérieur « Fichier_sortie » qui représente là où les données des grilles/générations seront sauvegardées.



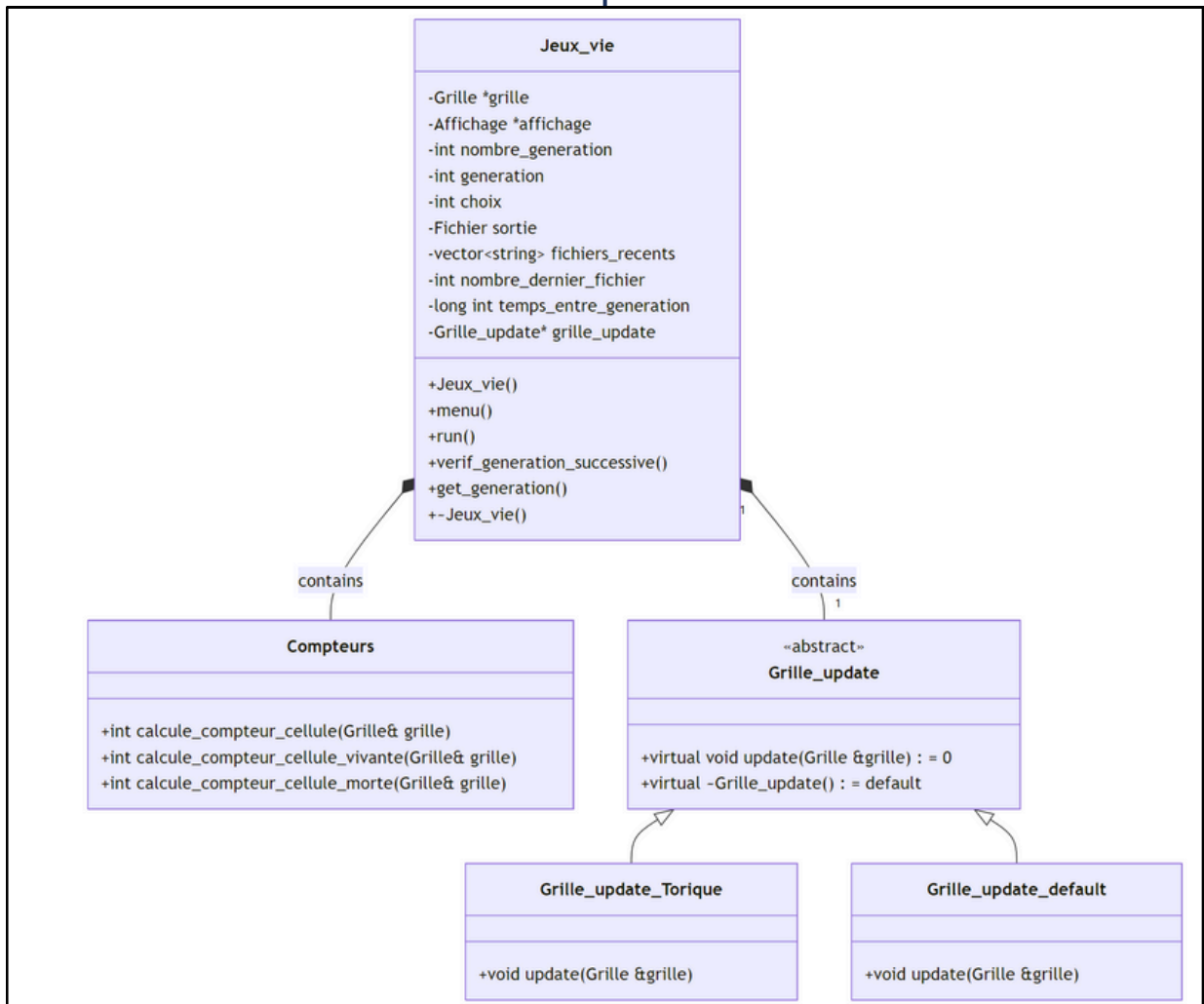
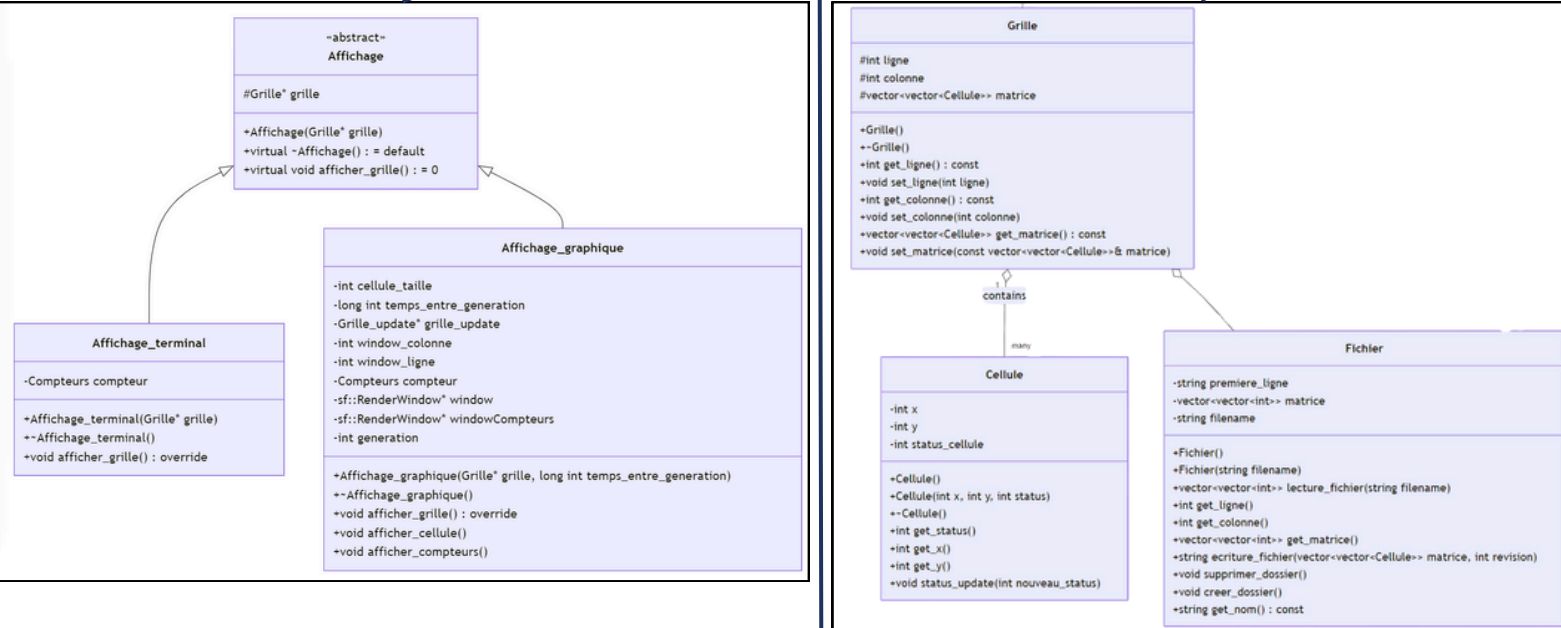
Diagramme de classe :

Les diagrammes de classes sont l'un des types de diagrammes UML les plus utiles, car ils décrivent clairement la structure d'un système particulier en modélisant ses classes, ses attributs, ses opérations et les relations entre ses objets. Avec notre logiciel de diagrammes UML, créer des diagrammes n'a jamais été aussi facile. Ce guide vous montrera comment comprendre, planifier et créer vos propres diagrammes de classes.



Pour plus d'informations sur les couches d'architecture logicielle, veuillez regarder le dernier annexe

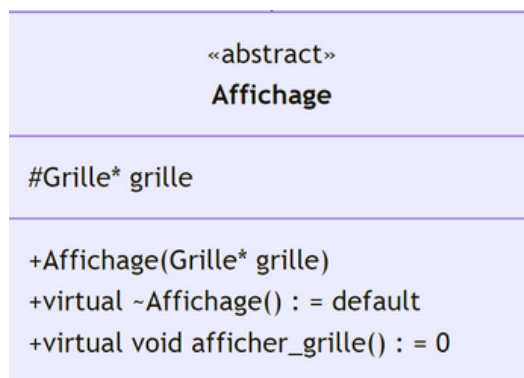
L'architecture logicielle du projet "Jeu de la Vie" peut être divisée en trois couches principales : la couche de présentation (UML), la couche métier (logiciel) et la couche de données.



Couche de Présentation (UML)

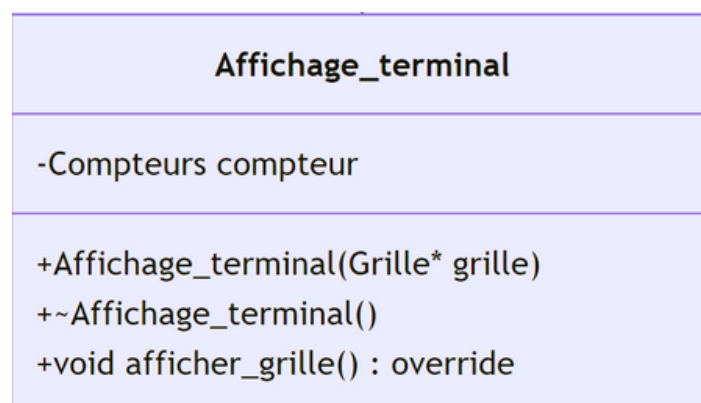
Cette couche est responsable de l'interaction avec l'utilisateur. Elle affiche les informations et recueille les entrées de l'utilisateur.

- Classes :
 - Affichage : Classe abstraite pour l'affichage de la grille.
 - Affichage_terminal : Affiche la grille dans le terminal.
 - Affichage_graphique : Affiche la grille graphiquement.
- Détails :
 - **Affichage** :
 - Description : Classe abstraite définissant l'interface pour l'affichage de la grille.
 - Attributs :
 - grille : Pointeur vers un objet Grille.
 - Méthodes :
 - Affichage(Grille* grille) : Constructeur avec paramètre.
 - ~Affichage() : Destructeur virtuel par défaut.
 - afficher_grille() : Méthode virtuelle pure pour afficher la grille.



Couche de Présentation (UML)

- **Affichage_terminal** :
 - Description : Implémentation de l'affichage de la grille dans le terminal.
 - Attributs :
 - compteur : Objet Compteurs pour calculer les compteurs.
 - Méthodes :
 - Affichage_terminal(Grille* grille) : Constructeur avec paramètre.
 - ~Affichage_terminal() : Destructeur.
 - afficher_grille() : Affiche la grille dans le terminal.



Couche de Présentation (UML)

- **Affichage_graphique :**

- Description : Implémentation de l'affichage de la grille graphiquement.
- Attributs :
 - cellule_taille : Taille des cellules.
 - temps_entre_generation : Temps entre chaque génération.
 - grille_update : Pointeur vers un objet Grille_update.
 - window_colonne : Nombre de colonnes de la fenêtre.
 - window_ligne : Nombre de lignes de la fenêtre.
 - compteur : Objet Compteurs pour calculer les compteurs.
 - window : Pointeur vers la fenêtre principale.
 - windowCompteurs : Pointeur vers la fenêtre des compteurs.
 - generation : Compteur de génération.
- Méthodes :
 - Affichage_graphique(Grille* grille, long int temps_entre_generation) : Constructeur avec paramètres.
 - ~Affichage_graphique() : Destructeur.
 - afficher_grille() : Affiche la grille graphiquement.
 - afficher_cellule() : Affiche les cellules.
 - afficher_compteurs() : Affiche les compteurs.

Affichage_graphique

-int cellule_taille
-long int temps_entre_generation
-Grille_update* grille_update
-int window_colonne
-int window_ligne
-Compteurs compteur
-sf::RenderWindow* window
-sf::RenderWindow* windowCompteurs
-int generation

+Affichage_graphique(Grille* grille, long int temps_entre_generation)
+~Affichage_graphique()
+void afficher_grille() : override
+void afficher_cellule()
+void afficher_compteurs()

Couche Métier (Logique d'Application) - Logiciel

Cette couche contient la logique principale du jeu, y compris les règles de mise à jour de la grille et la gestion des générations.

- Classes :
 - Jeux_vie : Classe principale du jeu.
 - Grille_update : Classe abstraite pour la mise à jour de la grille.
 - Grille_update_Torique : Implémentation de la mise à jour torique de la grille.
 - Grille_update_default : Implémentation de la mise à jour par défaut de la grille.
 - Compteurs : Classe pour calculer les compteurs de cellules vivantes et mortes.

Détails :

- **Jeux_vie** :
 - Description : Classe principale qui gère la grille, l'affichage, les générations et les fichiers.
 - Attributs :
 - grille : Pointeur vers un objet Grille.
 - affichage : Pointeur vers un objet Affichage.
 - nombre_generation : Nombre total de générations.
 - generation : Génération actuelle.
 - choix : Choix de l'utilisateur pour le type d'affichage.
 - sortie : Objet Fichier pour gérer les fichiers.
 - fichiers_recents : Vecteur de chaînes de caractères pour stocker les fichiers récents.
 - nombre_dernier_fichier : Nombre de fichiers récents à conserver.
 - temps_entre_generation : Temps entre chaque génération en millisecondes.
 - grille_update : Pointeur vers un objet Grille_update.
 - Méthodes :
 - Jeux_vie() : Constructeur.
 - menu() : Affiche le menu.
 - run() : Exécute le jeu.
 - verif_generation_successive() : Vérifie les générations successives.
 - get_generation() : Retourne la génération actuelle.
 - ~Jeux_vie() : Destructeur.

Jeux_vie

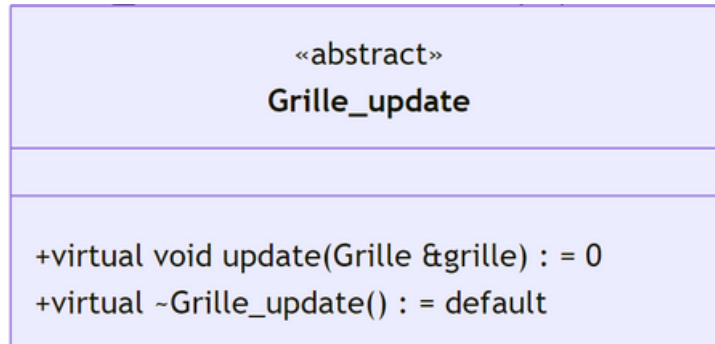
-Grille *grille
-Affichage *affichage
-int nombre_generation
-int generation
-int choix
-Fichier sortie
-vector<string> fichiers_recents
-int nombre_dernier_fichier
-long int temps_entre_generation
-Grille_update* grille_update

+Jeux_vie()
+menu()
+run()
+verif_generation_successive()
+get_generation()
+~Jeux_vie()

Couche Métier (Logique d'Application) - Logiciel

- **Grille_update :**

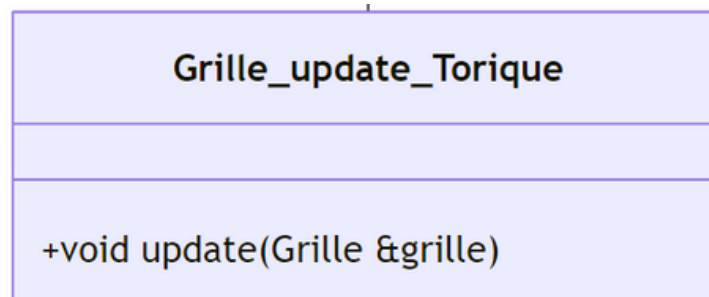
- Description : Classe abstraite définissant l'interface pour la mise à jour de la grille c'est un composant de la classe jeux_vie via le lien de composition.
- Méthodes :
 - update(Grille &grille) : Méthode virtuelle pure pour mettre à jour la grille.
 - ~Grille_update() : Destructeur virtuel par défaut.



Couche Métier (Logique d'Application) - Logiciel

- **Grille_update_Torique :**

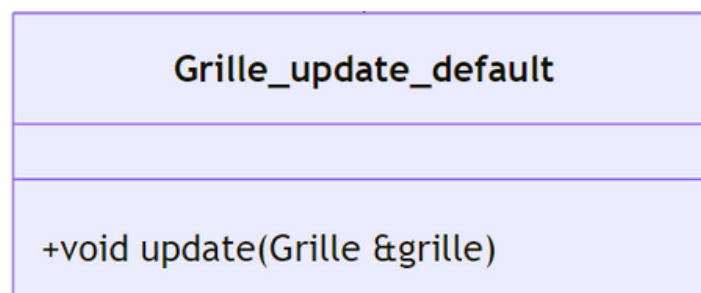
- Description : Implémentation de la mise à jour de la grille de manière torique.
- Méthodes :
 - update(Grille &grille) : Met à jour la grille de manière torique.



Couche Métier (Logique d'Application) - Logiciel

- **Grille_update_default :**

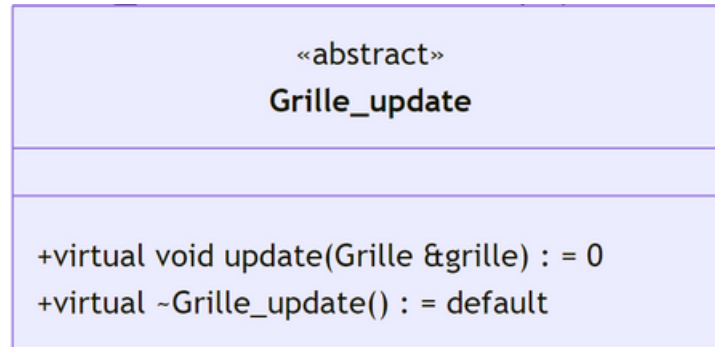
- Description : Implémentation de la mise à jour de la grille de manière par défaut.
- Méthodes :
 - update(Grille &grille) : Met à jour la grille de manière par défaut.



Couche Métier (Logique d'Application) - Logiciel

- **Grille_update :**

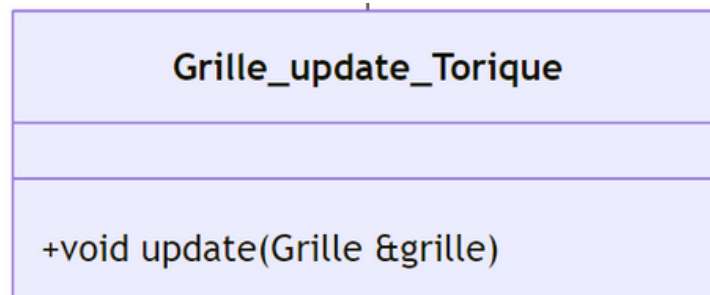
- Description : Classe abstraite définissant l'interface pour la mise à jour de la grille. C'est une classe composante de la classe jeux_vie via le lien de composition, c'est comme si les méthodes de la classe Grille se trouvaient dans celle de jeux_vie.
- Méthodes :
 - update(Grille &grille) : Méthode virtuelle pure pour mettre à jour la grille.
 - ~Grille_update() : Destructeur virtuel par défaut.



Couche Métier (Logique d'Application) - Logiciel

- **Grille_update_Torique :**

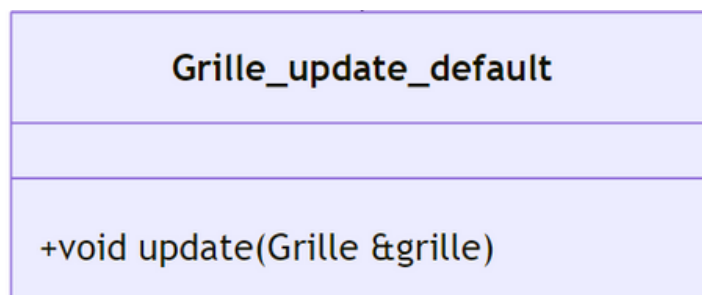
- Description : Implémentation de la mise à jour de la grille de manière torique.
- Méthodes :
 - update(Grille &grille) : Met à jour la grille de manière torique.



Couche Métier (Logique d'Application) - Logiciel

- **Grille_update_default :**

- Description : Implémentation de la mise à jour de la grille de manière par défaut.
- Méthodes :
 - update(Grille &grille) : Met à jour la grille de manière par défaut.



Couche Métier (Logique d'Application) - Logiciel

- **Compteurs :**

- Description : Classe pour calculer les compteurs de cellules vivantes et mortes dans la grille. C'est une classe composante de la classe jeux_vie via le lien de composition, c'est comme si les méthodes de la classe Grille se trouvaient dans celle de jeux_vie.
- Méthodes :
 - `calcule_compteur_cellule(Grille& grille)` : Calcule le nombre total de cellules.
 - `calcule_compteur_cellule_vivante(Grille& grille)` : Calcule le nombre de cellules vivantes.
 - `calcule_compteur_cellule_morte(Grille& grille)` : Calcule le nombre de cellules mortes.

Compteurs
<code>+int calcule_compteur_cellule(Grille& grille)</code> <code>+int calcule_compteur_cellule_vivante(Grille& grille)</code> <code>+int calcule_compteur_cellule_morte(Grille& grille)</code>

Couche de Données

Cette couche gère la persistance des données, y compris la lecture et l'écriture des fichiers de configuration de la grille.

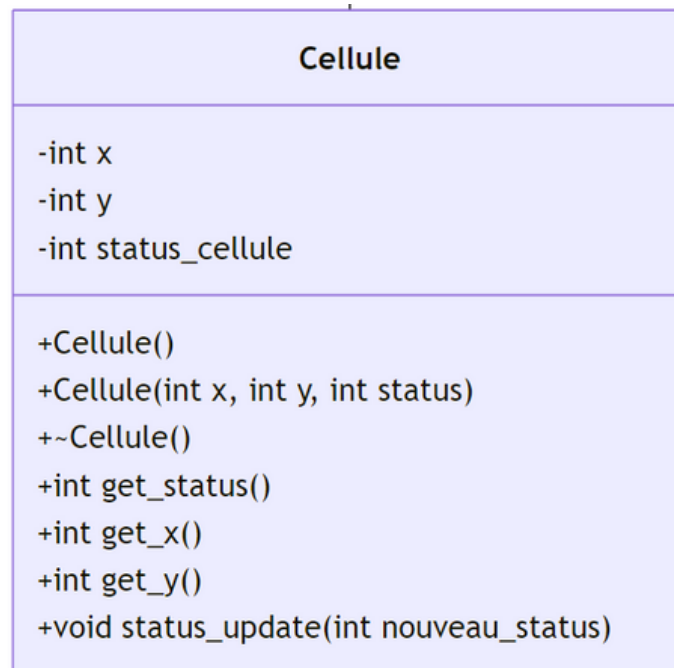
- Classes :
 - Grille : Représente la grille du jeu, contenant les cellules.
 - Cellule : Représente une cellule individuelle dans la grille.
 - Fichier : Gère la lecture et l'écriture des fichiers de configuration de la grille.
- Détails :
 - **Grille :**
 - Description : Cette classe représente la grille du jeu, contenant les cellules.
 - Attributs :
 - ligne : Nombre de lignes de la grille.
 - colonne : Nombre de colonnes de la grille.
 - matrice : Matrice de cellules.
 - Méthodes :
 - Grille() : Constructeur.
 - ~Grille() : Destructeur.
 - get_ligne() : Retourne le nombre de lignes.
 - set_ligne(int ligne) : Définit le nombre de lignes.
 - get_colonne() : Retourne le nombre de colonnes.
 - set_colonne(int colonne) : Définit le nombre de colonnes.
 - get_matrice() : Retourne la matrice de cellules.
 - set_matrice(const vector<vector<Cellule>>& matrice) : Définit la matrice de cellules

Grille
-int ligne -int colonne -vector<vector<Cellule>> matrice
+Grille() +~Grille() +int get_ligne() : const +void set_ligne(int ligne) +int get_colonne() : const +void set_colonne(int colonne) +vector<vector<Cellule>> get_matrice() : const +void set_matrice(const vector<vector<Cellule>>& matrice)

Couche de Données

- **Cellule :**

- Description : Cette classe représente une cellule individuelle dans la grille.
- Attributs :
 - x : Coordonnée x de la cellule.
 - y : Coordonnée y de la cellule.
 - status_cellule : Statut de la cellule (vivante ou morte).
- Méthodes :
 - Cellule() : Constructeur par défaut.
 - Cellule(int x, int y, int status) : Constructeur avec paramètres.
 - ~Cellule() : Destructeur.
 - get_status() : Retourne le statut de la cellule.
 - get_x() : Retourne la coordonnée x.
 - get_y() : Retourne la coordonnée y.
 - status_update(int nouveau_status) : Met à jour le statut de la cellule.



Couche de Données

- **Fichier :**

- Description : Cette classe gère la lecture et l'écriture des fichiers de configuration de la grille. Sans cette classe, on n'aura pas de grille initialisé.
- Attributs :
 - premiere_ligne : Première ligne du fichier contenant les dimensions.
 - matrice : Matrice de données lues du fichier.
 - filename : Nom du fichier.
- Méthodes :
 - Fichier() : Constructeur par défaut.
 - Fichier(string filename) : Constructeur avec paramètre.
 - lecture_fichier(string filename) : Lit un fichier.
 - get_ligne() : Retourne le nombre de lignes.
 - get_colonne() : Retourne le nombre de colonnes.
 - get_matrice() : Retourne la matrice de données.
 - ecriture_fichier(vector<vector<Cellule>> matrice, int revision) : Écrit dans un fichier.
 - supprimer_dossier() : Supprime un dossier.
 - creer_dossier() : Crée un dossier.
 - get_nom() const : Retourne le nom du fichier.

Fichier

-string premiere_ligne
-vector<vector<int>> matrice
-string filename

+Fichier()
+Fichier(string filename)
+vector<vector<int>> lecture_fichier(string filename)
+int get_ligne()
+int get_colonne()
+vector<vector<int>> get_matrice()
+string ecriture_fichier(vector<vector<Cellule>> matrice, int revision)
+void supprimer_dossier()
+void creer_dossier()
+string get_nom() : const

Diagramme de séquence:

Les diagrammes de séquence sont une solution populaire de modélisation dynamique en langage UML, car ils se concentrent plus précisément sur les lignes de vie, les processus et les objets qui vivent simultanément, et les messages qu'ils échangent entre eux pour exercer une fonction avant la fin de la ligne de vie. Parallèlement à notre outil de création de diagrammes UML, utilisez ce guide pour tout savoir sur les diagrammes de séquence en langage UML.

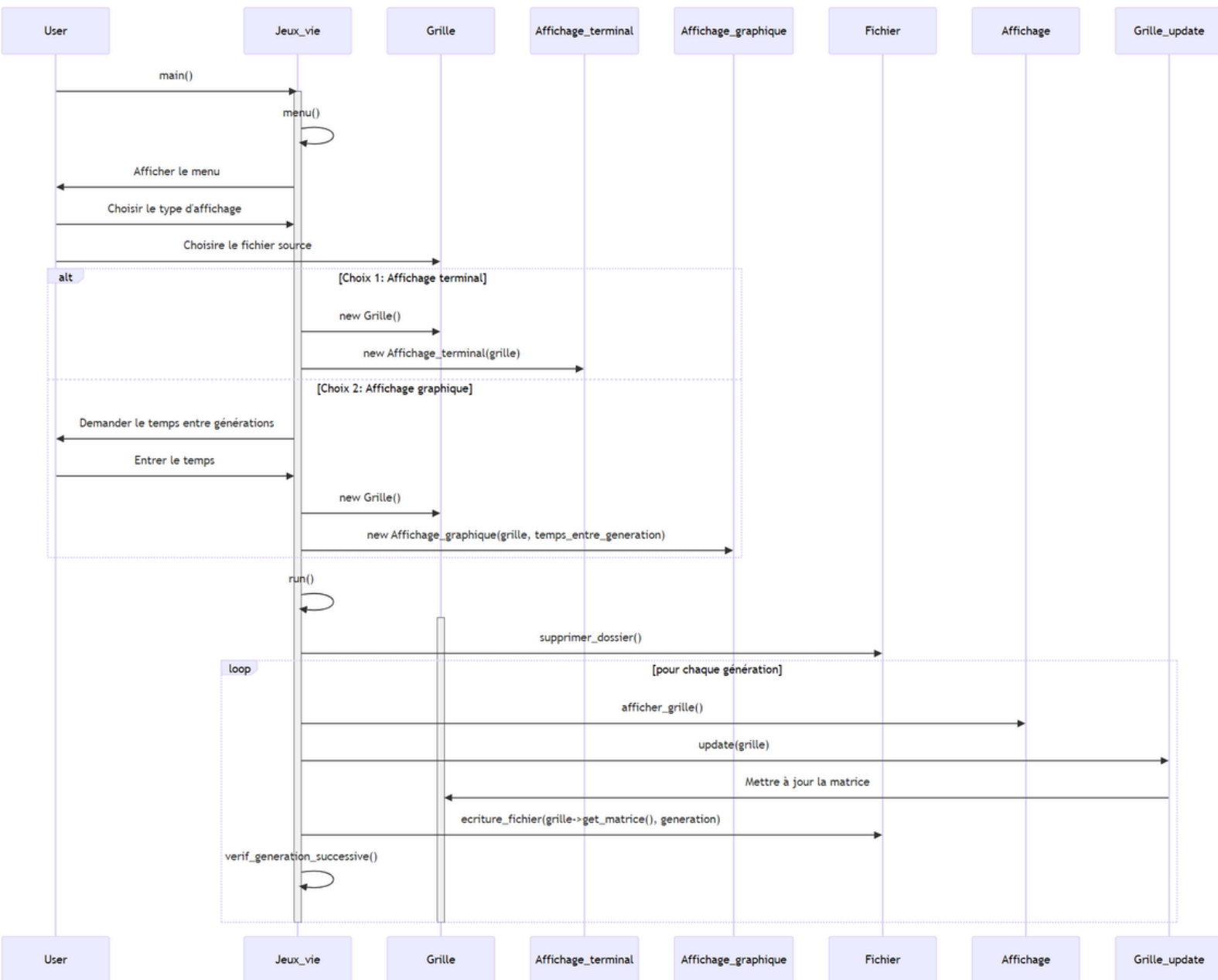


Diagramme de séquence:

Participants

- Participants
- User : L'utilisateur qui interagit avec le programme.
- Jeux_vie : La classe principale qui gère le déroulement du jeu.
- Grille : La classe représentant la grille du jeu, contenant les cellules.
- Affichage_terminal : La classe responsable de l'affichage de la grille dans le terminal.
- Affichage_graphique : La classe responsable de l'affichage graphique de la grille.
- Affichage : Une classe abstraite ou interface pour les différentes méthodes d'affichage.
- Grille_update : La classe responsable de la mise à jour de la grille selon les règles du jeu.
- Fichier : La classe responsable de la gestion des fichiers (lecture et écriture).

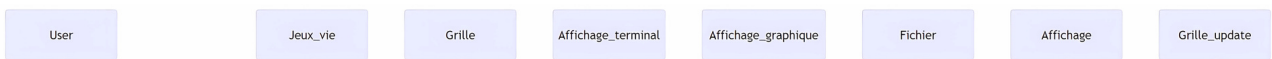


Diagramme de séquence:

Description du Diagramme de Séquence

1. Initialisation du Jeu :

- L'utilisateur lance le programme en appelant la fonction `main()` qui elle crée un objet jeux de vie
- La méthode `menu()` de `Jeux_vie` est appelée pour afficher le menu à l'utilisateur.
- L'utilisateur choisit le type d'affichage (terminal ou graphique) et sélectionne le fichier source pour initialiser la grille.

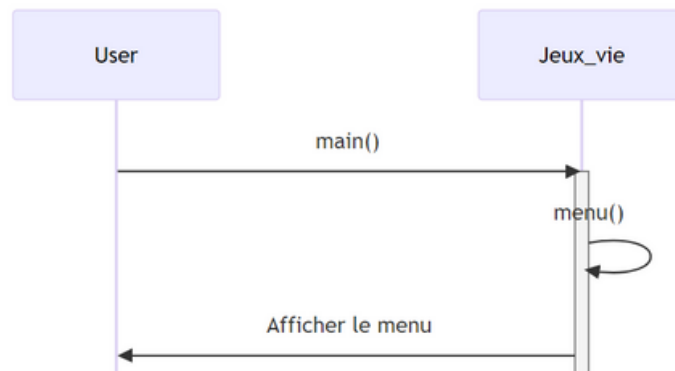


Diagramme de séquence:

2.Choix de l’Affichage :

- Affichage Terminal :
 - Si l'utilisateur choisit l'affichage terminal, une nouvelle instance de Grille est créée.
 - Une nouvelle instance de Affichage_terminal est créée avec la grille comme paramètre.
- Affichage Graphique :
 - Si l'utilisateur choisit l'affichage graphique, le programme demande à l'utilisateur d'entrer le temps entre chaque génération.
 - Une nouvelle instance de Grille est créée.
 - Une nouvelle instance de Affichage_graphique est créée avec la grille et le temps entre les générations comme paramètres.

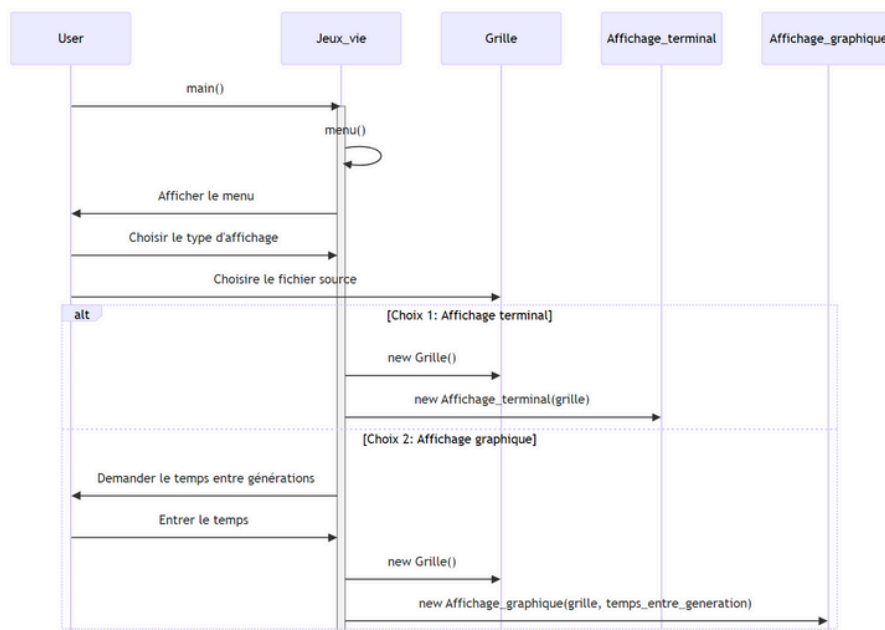


Diagramme de séquence:

3.Exécution du Jeu :

- La méthode run() de Jeux_vie est appelée pour démarrer le jeu.
- La méthode supprimer_dossier() de Fichier est appelée pour supprimer les anciens fichiers de sortie.
- Pour chaque génération :
 - La méthode afficher_grille() de Affichage (terminal ou graphique) est appelée pour afficher la grille.
 - La méthode update() de Grille_update est appelée pour mettre à jour la grille selon les règles du jeu.
 - La méthode ecriture_fichier() de Fichier est appelée pour sauvegarder l'état actuel de la grille dans un fichier.
 - La méthode verif_generation_successive() de Jeux_vie est appelée pour vérifier si les générations successives sont identiques.

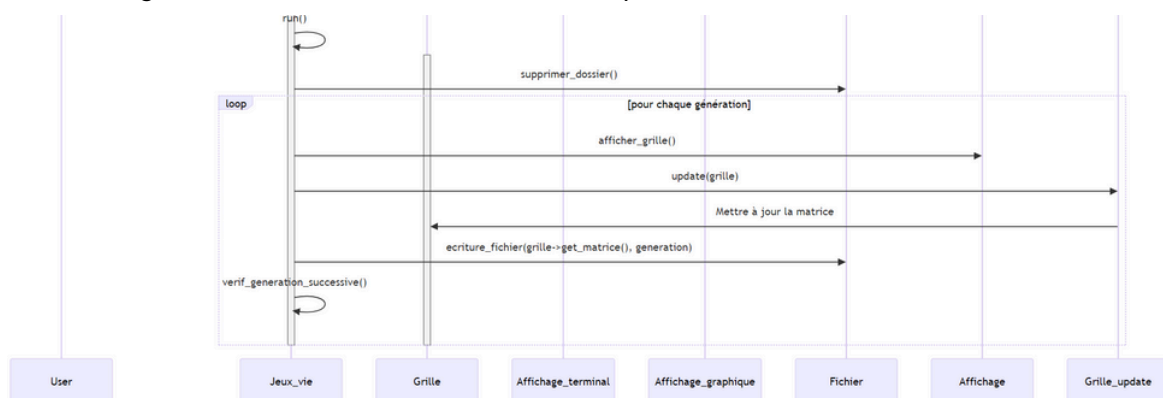
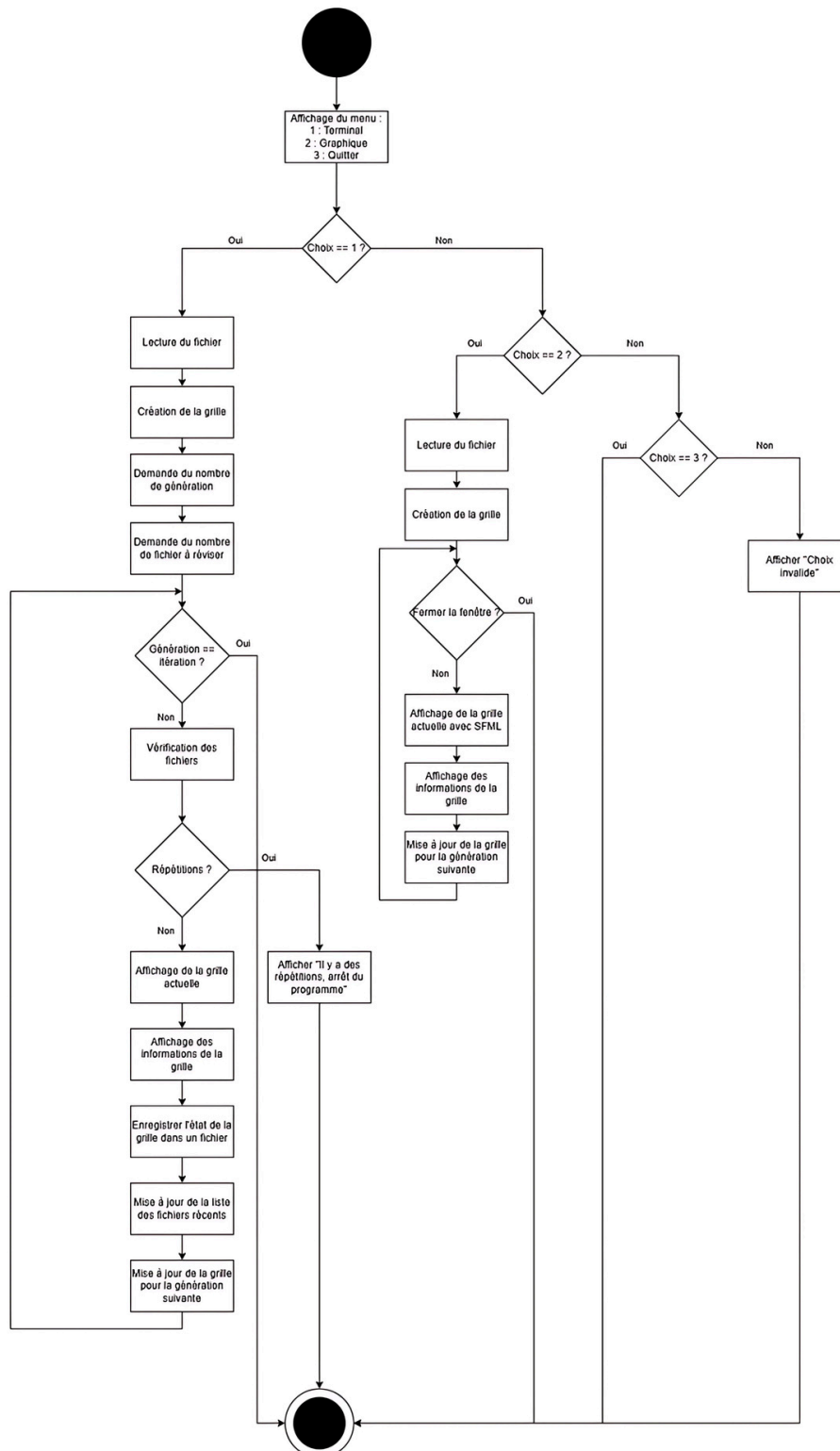


Diagramme d'activité :

Un diagramme d'activité est un diagramme UML qui illustre les étapes d'un processus ou d'une opération en modélisant les flux de contrôle et les décisions. Les diagrammes d'activité décrivent la séquence d'activités, les transitions, les actions parallèles ou alternatives, et les conditions associées. Ces diagrammes aident les équipes à comprendre comment les tâches se déroulent dans un système, mettant en évidence les dépendances et les points de synchronisation. Un diagramme d'activité peut représenter un flux global dans un système ou se concentrer sur un processus spécifique, permettant ainsi de modéliser des comportements dynamiques et d'identifier d'éventuels points d'amélioration.



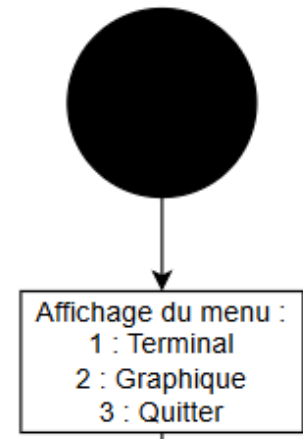
Interaction utilisateur :

Ce diagramme décrit le fonctionnement général de notre système.

Le programme commence donc par l'affichage d'un menu interactif permettant à l'utilisateur de faire un choix entre trois possibilités :

1. Terminal : permet une interaction via la console
2. Graphique : permet une interaction avec l'interface graphique
3. Quitter : met fin au programme

Le déroulement du programme dépendra du choix de l'utilisateur.



Scénario n°1 Choix = 1:

Le choix du terminal

1. Lecture du fichier : le programme commence par lire un fichier contenant les données initiales (état de la grille).
2. Création de la grille : une grille de cellules est générée à partir des données du fichier.
3. Demande du nombre de générations : l'utilisateur entre le nombre de générations à simuler.
4. Demande du nombre de fichiers à réviser : le programme demande à l'utilisateur combien de fichiers il souhaite réviser.
5. Boucle principale
 - A. Génération : si une génération est valide, le programme passe à la vérification des fichiers, sinon la boucle s'arrête.
 - B. Vérification des fichiers : le programme vérifie si des répétitions sont détectées dans l'état des grilles
 - C. Répétitions : si des répétitions sont détectées, le programme affiche un message pour prévenir l'utilisateur et s'arrête. Sinon, il affiche la grille actuelle, les informations de la grille, enregistre l'état de la grille dans un fichier, met à jour la liste des fichiers à réviser et enfin, il met à jour la grille.

Ce processus boucle jusqu'à ce que toutes les générations soient simulées ou qu'une répétition soit détectée.

Scénario n°2 Choix = 2:

Le choix de l'affichage graphique (Choix == 2)

1. Lecture du fichier : comme dans le mode Terminal, les données initiales sont chargées depuis un fichier
2. Création de la grille : une grille est générée à partir des données.
3. Boucle principale
 - A. Si l'utilisateur ferme la fenêtre, le programme s'arrête.
 - B. Sinon : la grille est affichée à l'aide de SFML, on affiche les informations de la grille et on met à jour la grille pour la prochaine génération

Scénario n°3 Choix = 3:

Le choix de Quitter

Le programme s'arrête tout de suite.

Le choix est invalide si l'utilisateur entre une valeur incorrecte (différente de 1, 2 ou 3), le programme affiche le message : « Choix invalide » et arrête le programme.

IMPLÉMENTATION DU CODE

Le fichier Makefile

Tout d'abord nous avons le fichier « Makefile » qui permet de compiler l'intégralité du projet. Il a été conçu en utilisant la bibliothèque SFML. Il inclut des cibles pour la compilation standard, la compilation en mode débogage et le nettoyage des fichiers générés.

Cibles principales :

All : main :

Cette cible permet de compiler le projet en générer un exécutable nommé main en utilisant les fichiers source .cpp et les fichiers d'en-tête .h détectés dans le répertoire du projet.

Main : cette cible produit un exécutable en mode normal avec les options suivantes :

1. Inclut les bibliothèques SFML nécessaire (graphics, window, audio, network, system).
2. Active les options de débogage basique (-g).
3. Active des avertissements (-Wmost - Werror).

Main-debug :

Cette cible produit un exécutable en mode débogage avec les options supplémentaires suivantes :

1. Désactive l'optimisation pour faciliter le débogage (-O0).
2. Supprimer l'activation de la protection _FORTIFY_SOURCE.

Clean

Supprimer les fichiers exécutables générés (main et main-debug).

Variables utilisées :

CXX : définit le compilateur utilisé, ici clang++.

CXXFLAGS : contient les options de compilation

1. Liens vers les bibliothèques SFML.
2. Activation du débogage (-g) et des avertissements stricts.

SRCS : Liste automatiquement tous les fichiers source .cpp en excluant les répertoires de cache .ccls-cache).

HEADERS : liste automatiquement tous les fichiers d'en-tête .h en excluant également les répertoire de cache.

La classe Fichier

Cette classe gère la lecture, l'écriture et la gestion des fichiers dans un projet. La classe permet de manipuler des matrices stockées dans des fichiers texte et de sauvegarder des données formatées.

Fichier
-string premiere_ligne -vector<vector<int>> matrice -string filename
+Fichier() +Fichier(string filename) +vector<vector<int>> lecture_fichier(string filename) +int get_ligne() +int get_colonne() +vector<vector<int>> get_matrice() +string ecriture_fichier(vector<vector<Cellule>> matrice, int revision) +void supprimer_dossier() +void creer_dossier() +string get_nom() : const

Cette classe est divisée en deux fichiers :

Fichier.h : qui contient la définition de la classes avec les attributs et les méthodes.

Fichier.cpp : qui contient l'implémentation des méthodes définies précédemment.

Les attributs privés sont les suivants :

String premiere_ligne : stocke la premier ligne du fichier qui correspond aux dimensions de la matrice.

Vector<vecotr<int>> matrice : stocke les données matricielles lues dans le fichier.

String filename : stocke le nom du fichier associé à l'instance

Les méthodes publiques sont les suivantes :

- Constructeurs

Fichier() : constructeur par défaut.

Fichier(string filename) : Constructeur paramétré pour initialiser un fichier avec un nom donné.

- Lecture de fichier

vector<vector<int>> lecture_fichier(string filename) : méthode qui permet de lire une matrice, la première ligne doit spécifier les dimensions de la matrice (lignes et colonnes).

La méthode retourne un vecteur 2D contenant les données du fichier et gère les erreurs d'ouverture du fichier.

- Accesseurs

Int get_ligne() : retourne le nombre de lignes de la matrice.

Int get_colonne() : retourne le nombre de colonnes de la matrice.

vector<vector<int>> get_matrice() : retourne la matrice.

String get_nom() const : retourne le nom du fichier.

- Écriture du fichier

string ecriture_fichier(vector<vector<Cellule>> matrice, int revision) : sauvegarde une matrice dans un fichier texte nommé dynamiquement selon une révision donnée, écrit les dimensions de la matrice en première ligne, écrit les états des cellules de la matrice (via **Cellule::get_status()**) et retourne le nom du fichier créé pour suivi.

- Gestion des dossier

void supprimer_dossier() : supprimer tout le contenu du dossier **Sortie_sauvegarde** en appelant une commande système.

void creer_dossier() : permet de créer le dossier **Sortie_sauvegarde** s'il n'existe pas.

Utilisation

Pour lire une matrice depuis un fichier :

```
Fichier fichier("mon_fichier.txt");  
vector<vector<int>> matrice = fichier.lecture_fichier("mon_fichier.txt");
```

Écriture d'une matrice :

```
Fichier fichier;  
// Matrice à sauvegarder  
vector<vector<int>> matrice = fichier.lecture_fichier("mon_fichier.txt");  
string fichier_sortie = fichier.ecriture_fichier(matrice, 1);  
cout << "Matrice sauvegardée dans : " << fichier.sortie << endl;
```

Gestion des dossiers : pour réinitialiser le dossier des sauvegardes :

```
Fichier fichier;  
fichier.supprimer_dossier();  
fichier.creer_dossier()
```

Points importants :

Structure de fichier attendu : le fichier à lire doit respecter un format strict avec les dimensions en première ligne suivie des données matricielles.

Gestion des erreurs :

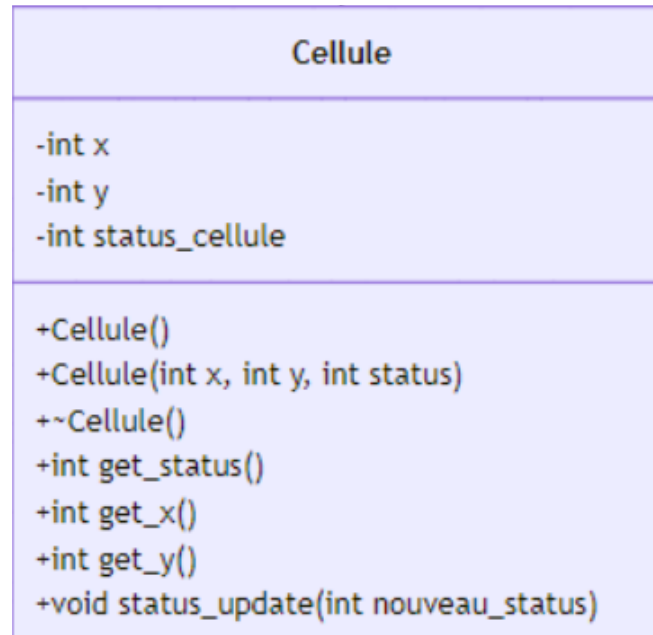
Les ouvertures de fichier sont vérifiées pour éviter les plantages.

En cas d'échec, un message d'erreur est affiché, et des valeurs par défaut (ou chaîne vide) sont retournées.

Commandes système : les méthodes **supprimer_dossier** et **creer_dossier** utilisent **system** pour exécuter des commandes linux. Pour cela, il faut vérifier si l'environnement d'exécution est compatible.

La classe Cellule

Cette classe représente une cellule d'une grille avec ses coordonnées et son état. Elle est utilisée dans des applications manipulant des structures matricielles, comme des grilles ou des tableaux 2D.



Elle est divisée en deux fichiers :

Cellule.h : qui contient la définition de la classes avec les attributs et les méthodes.

Cellule.cpp : qui contient l'implémentation des méthodes définies précédemment.

Les attributs privés sont les suivants :

int x, int y : pour les coordonnées de la cellule.

int status_cellule : stocke le statut de la cellule.

Les méthodes publiques sont les suivantes :

- Constructeur et destructeur :

Cellule() : constructeur par défaut qui initialise une cellule avec coordonnées (0, 0) et état initial 0 (statut par défaut).

Cellule(int x, int y, int status) : constructeur paramétré pour initialiser une cellule avec les coordonnées **x** et **y** et un état initial **status**.

~Cellule() : destructeur, utilisé pour nettoyer les ressources associées à une cellule si nécessaire.

- Accesseurs :

int get_status() : retourne l'état actuel de la cellule (**status_cellule**).

int get_x() : retourne la coordonnée x de la cellule.

int get_y() : retourne la coordonnées y de la cellule.

- Mise à jour :

Void status_update(int nouveau_status) : met à jour l'état de la cellule avec une nouvelle valeur (**nouveau_status**).

Utilisation

Créer une cellule par défaut :

```
Cellule cellule;  
cout << "Coordonnées : (" << cellule.get_x() << ", " << cellule.get_y() << ")"  
<< endl;  
cout << "État initial : " << cellule.get_status() << endl;
```

Créer une cellule personnalisée :

```
Cellule cellule(2, 3, 1);  
cout << "Coordonnées : (" << cellule.get_x() << ", " << cellule.get_y() << ")"  
<< endl;  
cout << "État initial : " << cellule.get_status() << endl;
```

Mettre à jour l'état d'une cellule :

```
Cellule cellule(1, 1, 0);  
cellule.status_update(1);  
cout << "Nouvel état : " << cellule.get_status() << endl;
```

Points importants :

Modèle de données simple : une cellule contient uniquement des informations sur ses coordonnées et son état. Elle peut être facilement intégrée dans des structures plus grandes comme une matrice.

Flexibilité : les méthodes **get_x**, **get_y**, **get_status** permettent de récupérer les données, tandis que **status_update** facilite la modification de l'état.

Portabilité : la classe est simple et légère, sans dépendances externes, ce qui la rend facile à utiliser dans différents projets.

La classe Grille

Cette classe représente une grille bidimensionnelle composée de cellules. Cette grille est initialisée à partir d'un fichier, permettant de gérer des structures matricielles et d'organiser les données sous forme d'objets Cellule.

Grille
<pre>#int ligne #int colonne #vector<vector<Cellule>> matrice</pre>
<pre>+Grille() +~Grille() +int get_ligne() : const +void set_ligne(int ligne) +int get_colonne() : const +void set_colonne(int colonne) +vector<vector<Cellule>> get_matrice() : const +void set_matrice(const vector<vector<Cellule>>& matrice)</pre>

Elle est divisée en deux fichiers :

Grille.h : qui contient la définition de la classes avec les attributs et les méthodes.

Grille.cpp : qui contient l'implémentation des méthodes définies précédemment.

Les attributs protégés sont les suivants :

Int ligne, int colonne : stocke le nombre de ligne et de colonne.

Vector<vector<Cellule>> matrice : cette variable stocke une matrice bidimensionnelle de d'instance de Cellule.

Les méthodes publiques sont les suivantes :

- Constructeur et destructeur :

Grille() : constructeur par défaut qui demande le nom d'un fichier en entrée utilisateur, utilise un objet de type Fichier pour lire les dimensions et les valeurs de la matrice contenue dans le fichier et initialise la grille en créant une matrice de cellules basée sur les données extraites.

~Grille() : destructeur qui nettoie les ressources associées à une instance de Grille.

- Accesseurs :

Int get_ligne() const : retourne le nombre de lignes de la grille.

Int get_colonne() const : retourne le nombre de colonne de la grille.

Vector<vector<Cellule>> get_matrice() const : retourne la matrice de cellules sous la forme d'un vecteur 2D.

- Mutateurs :

Void set_ligne(int ligne) : met à jour le nombre de lignes de la grille.

Void set_colonne(int colonne) : met à jour le nombre de colonne de la grille.

Void set_matrice(const vector<vector<Cellule>>& matrice) : met à jour la matrice avec une nouvelle structure. Utilisation d'un passage par référence de la matrice pour modifier la variable d'origine et ne pas créer de copie.

Utilisation

Créer une grille depuis un fichier :

```
Grille grille; // L'utilisateur entre le nom du fichier lors de l'exécution
cout << "Nombre de lignes : " << grille.get_ligne() << endl;
cout << "Nombre de colonnes : " << grille.get_colonne() << endl;
```

Accéder aux cellules de la matrice :

```
auto matrice = grille.get_matrice();
for (const auto& ligne : matrice) {
    for (const auto& cellule : ligne) {
        cout << cellule.get_status() << " ";
    }
    cout << endl;
}
```

Modifier la grille :

```
grille.set_ligne(10); // Changer le nombre de lignes
grille.set_colonne(5); // Changer le nombre de colonnes

// Mettre à jour la matrice avec une nouvelle structure
std::vector<std::vector<Cellule>> nouvelle_matrice(10, std::vector<Cellule>(5,
Cellule(0, 0, 0)));
grille.set_matrice(nouvelle_matrice);
```

Points importants :

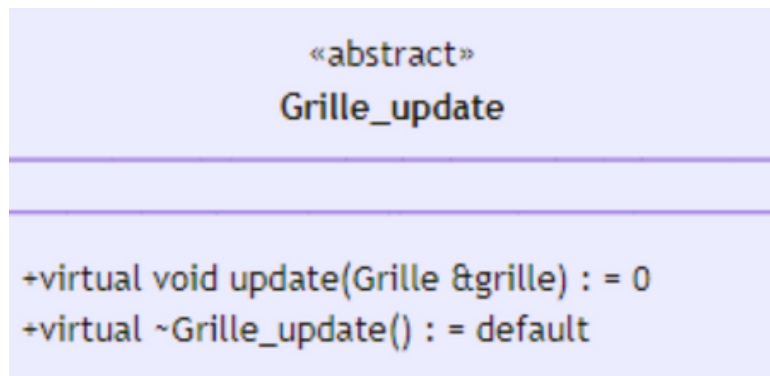
Initialisation automatique : la grille est automatiquement créée à partir d'un fichier donnée par l'utilisateur, rendant le processus transparent.

Structure organisée : la grille est composée d'objets **Cellules**, ce qui permet une manipulation orientée objet des données.

Flexibilité : les accesseurs et mutateurs permettent de lire et de modifier la structure de la grille facilement.

La classe Grille_update

Le fichier Grille_update.h définit une classe abstraite destinée à fournir une interface pour mettre à jour les objets de type Grille. Il établit une structure permettant de définir différentes stratégies ou méthodes d'actualisation des données dans une grille.



La seule méthode publique est la méthode **virtuelle pure update()**.

virtual void update(Grille &grille) = 0;

Cette méthode devra donc être réimplémentée dans toutes les classes dérivées. De plus, elle prend une référence à un objet Grille en argument, permettant ainsi la modification directe de cet objet.

Points importants :

Passage par référence : la méthode update utilise une référence (**Grille &grille**) pour manipuler directement l'objet passé en paramètre sans en créer de copie.

Flexibilité et extensibilité : en une classe abstraite, ce design permet d'ajouter facilement de nouvelles logiques de mise à jour en créant des classes dérivées sans modifier le code existant.

Modularité : la séparation de la logique de mise à jour dans une classe dédiée améliore la lisibilité.

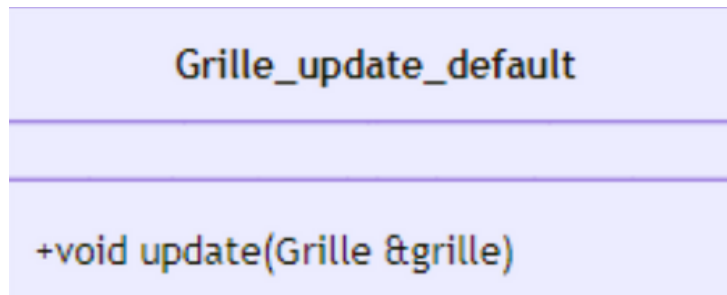
Avantages :

Réutilisabilité : les classes dérivées peuvent être utilisées dans différents contextes, tant qu'elles respectent l'interface définie.

Respect des principes SOLID : l'approche respecte notamment le principe de substitution de Liskov.

La classe **Grille_update_default**

La classe **Grille_update_default** est une implémentation concrète de la classe abstraite **Grille_update**. Elle applique un algorithme de mise à jour des cellules d'une grille en fonction de leurs voisins.



Elle est divisée en deux fichiers :

Grille_update_default.h : qui contient la définition de la classes avec les attributs et les méthodes.

Grille_update_default.cpp : qui contient l'implémentation des méthodes définies précédemment.

Cette classe hérite donc publiquement de la classe **Grille_update**.

Elle fournit une méthode **update** qui implémente la logique de mise à jour pour les cellules d'une grille.

void update(Grille &grille);

Fonctionnement de la méthode :

La méthode reçoit une référence à une grille (**Grille &grille**)

Elle récupère les dimensions et la matrice de cellules via les méthodes **get_matrice**, **get_ligne** et **get_colonne**.

Une copie de la matrice, appelée **nouvelle_matrice**, est créée pour stocker les états mis à jour.

Chaque cellule de la matrice est mise à jour en fonction des règles suivantes :

Règle 1 : Une cellule vivante (statut 1) avec moins de 2 ou plus de 3 voisins vivants meurt (statut 0).

Règle 2 : Une cellule morte (statut 0) avec exactement 3 voisins vivants devient vivantes (statut 1).

Règle 3 : Les cellules de statut 2 ne sont pas modifiées (extension du « Jeu de la Vie »).

Mise à jour des voisins :

L'algorithme parcourt les 8 voisins immédiats d'une cellule.

Les voisins hors des limites de la matrice sont ignorés.

Finalisation :

La grille est mise à jour avec la nouvelle matrice en appelant **grille.set_matrice(nouvelle_matrice)**.

Utilisation

Mise à jour d'une grille :

```
Grille maGrille;  
Grille_update_default miseAJour;  
miseAJour.update(maGrille);
```

Points importants :

Passage par référence : la méthode reçoit l'objet **Grille** par référence, ce qui permet de la modifier directement sans créer de copie inutile.

Modularité : en tant qu'implémentation d'une classe abstraite, cette classe peut être remplacée par d'autres version, sans modifier le code principal.

Optimisation :

Une matrice temporaire est utilisée pour éviter de modifier directement l'état des cellules pendant que les voisins sont encore évalués.

Avantages :

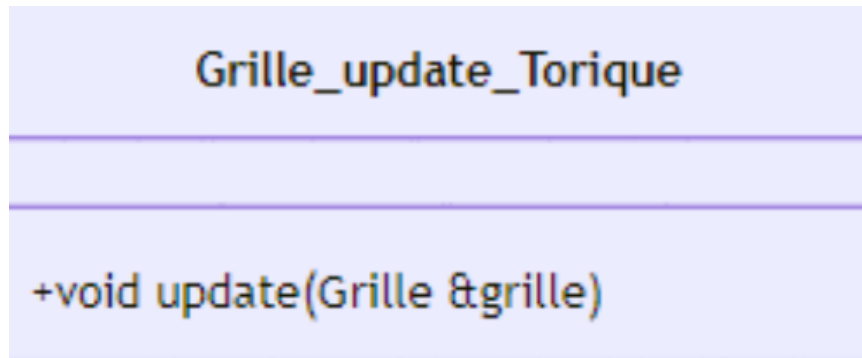
Facilité de compréhension : les règles sont implémentées de manière claire et directement liées aux mécanismes des automates cellulaires.

Extensibilité : de nouvelles règles de mise à jour peuvent être ajoutées en implémentant d'autres classes héritant de **Grille_update**.

Performance : l'algorithme est optimisé pour éviter les itérations inutiles en utilisant des conditions et des limites claires.

La classe Grille_update_Torique

La classe Grille_update_Torique est une implémentation concrète de la classe abstraite Grille_update. Contrairement à Grille_update_default, cette classe utilise une grille torique. Cela signifie que les cellules sur les bords de la grille sont connectées aux cellules opposées, formant ainsi un espace continu.



Elle est divisée en deux fichiers :

Grille_update_Torique.h : qui contient la définition de la classes avec les attributs et les méthodes.

Grille_update_Torique.cpp : qui contient l'implémentation des méthodes définies précédemment.

Cette classe hérite donc publiquement de **Grille_update**. Elle implémente la méthode virtuelle pure update pour gérer une grille torique.

void update(Grille &grille);

Fonctionnement :

Elle récupère la matrice et ses dimensions (ligne et colonne) via les méthodes de la classe Grille.

Crée une copie temporaire de la matrice (**nouvelle_matrice**) pour stocker les états mis à jour sans interférer avec l'état actuel.

Logique de mise à jour :

Grille torique :

Les indices des voisins sont calculés en utilisant l'opération **modulo** (%), permettant de relier les bords opposés de la grille.

Exemple : si i est à la dernière ligne et qu'un voisin est à $i + 1$, l'opération $(i + 1) \% \text{ligne}$ renvoie 0, reliant ainsi le bas au haut.

Chaque cellule de la matrice est mise à jour en fonction des règles suivantes :

Règle 1 : Une cellule vivante (statut 1) avec moins de 2 ou plus de 3 voisins vivants meurt (statut 0).

Règle 2 : Une cellule morte (statut 0) avec exactement 3 voisins vivants devient vivantes (statut 1).

Règle 3 : Les cellules de statut 2 ne sont pas modifiées (extension du « Jeu de la Vie »).

Finalisation :

La grille est mise à jour avec la nouvelle matrice en appelant **grille.set_matrice(nouvelle_matrice)**.

Utilisation

Mise à jour d'une grille :

```
Grille maGrille;  
Grille_update_Torique miseAJourTorique;  
miseAJourTorique.update(maGrille);
```

Points importants :

Grille torique : l'utilisation de l'opérateur **modulo** permet de connecter les bords opposés de la grille.

Cela simule un espace en forme de cylindre ou de sphère.

Passage par référence :

La méthode reçoit une référence à la grille (**Grille &grille**), ce qui permet de la modifier directement sans duplication inutile.

Modularité :

Cette implémentation peut être remplacée ou complétée par d'autres variantes sans affecter le reste du code, grâce au **polymorphisme**.

Séparation des responsabilités :

La logique de mise à jour est encapsulée dans cette classe, favorisant une architecture claire et extensible.

Avantages :

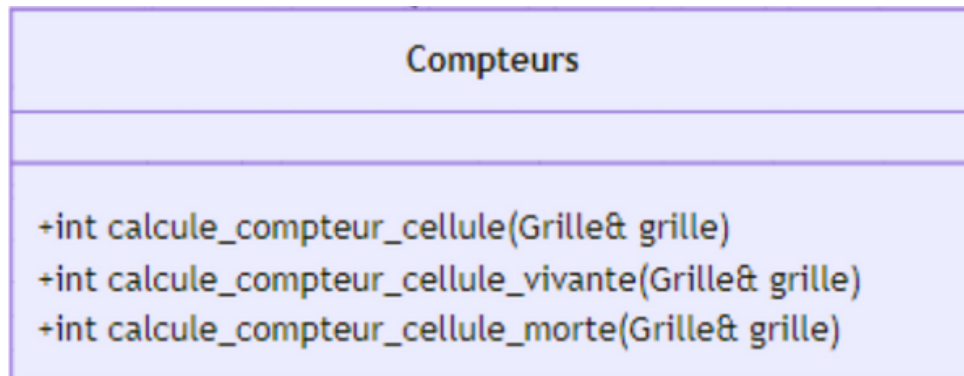
Gestion des bords : les cellules aux bords de la grilles sont mises à jour de manière cohérente avec celles des bords opposés.

Extensibilité : compatible avec d'autres implémentations héritant de **Grille_update**.

Efficacité : l'algorithme parcourt la grille avec une complexité en **$O(n * m)$** , où n et m sont respectivement les dimensions de la grille.

La classe Compteurs

La classe Compteur est responsable du calcul des statistiques relatives à l'état des cellules de la grille du jeu. Elle fournit des méthodes pour compter le nombre total de cellules, ainsi que les cellules vivantes et mortes dans la grille.



Elle est divisée en deux fichiers :

Compteurs.h : qui contient la définition de la classes avec les attributs et les méthodes.

Compteurs.cpp : qui contient l'implémentation des méthodes définies précédemment.

En voici les méthodes publiques :

int calcule_compteur_cellule(Grille& grille);

Celle-ci calcule le nombre total de cellules dans la grille en multipliant le nombre de ligne par le nombre de colonnes. Elle prend en paramètre une référence vers une instance la classe **Grille**.

int calcule_compteur_cellule_vivante(Grille& grille);

Celle-ci calcule le nombre de cellules vivantes dans la grille. Une cellule est considérée vivante si son état est égal à 1. Elle prend en paramètre une référence vers une instance la classe **Grille**.

int calcule_compteur_cellule_morte(Grille& grille);

Celle-ci calcule le nombre de cellules mortes dans la grille. Une cellule est considérée morte si son état est égal à 0. Elle prend en paramètre une référence vers une instance la classe **Grille**.

Utilisation

Compter le nombre total de cellule :

```
Compteurs compteur;  
int total_cellules = compteur.calculer_compteur_cellule(ma_grille);
```

Compter le nombre de cellule vivante :

```
Compteurs compteur;  
int cellules_vivantes = compteur.calculer_compteur_cellule_vivante(ma_grille);
```

Compter le nombre de cellule morte :

```
Compteurs compteur;  
int cellules_mortes = compteur.calculer_compteur_cellule_morte(ma_grille);
```

Détail d'implémentation :

Calcul du nombre de total de cellules :

La méthode **calculer_compteur_cellule()** utilise les méthodes **get_ligne()** et **get_colonne()** de la classe **Grille** pour obtenir les dimensions de la grille et retourne leur produit.

Calcul des cellules vivantes :

La méthode **calculer_compteur_cellule_vivante()** parcourt la matrice de la grille, en vérifiant le statut de chaque cellule. Si le statut est 1 elle incrémente le compteur associé.

Calcul des cellules mortes :

La méthode **calculer_compteur_cellule_morte()** parcourt la matrice de la grille, en vérifiant le statut de chaque cellule. Si le statut est 0 elle incrémente le compteur associé.

Points importants :

Méthodes indépendantes : chaque méthode de la classe **Compteurs** effectue une tâche spécifique et peut être utilisée indépendamment des autres.

Utilisation de la classe **Grille** : la classe **Compteurs** interagit directement avec la classe **Grille** pour accéder à ses dimensions et à son état interne, ce qui facilite le calcul des statistiques.

Simplicité d'utilisation : ces méthodes sont faciles à intégrer dans n'importe quel jeu de la vie, en fournissant des informations détaillées sur l'état de la grille.

Avantages :

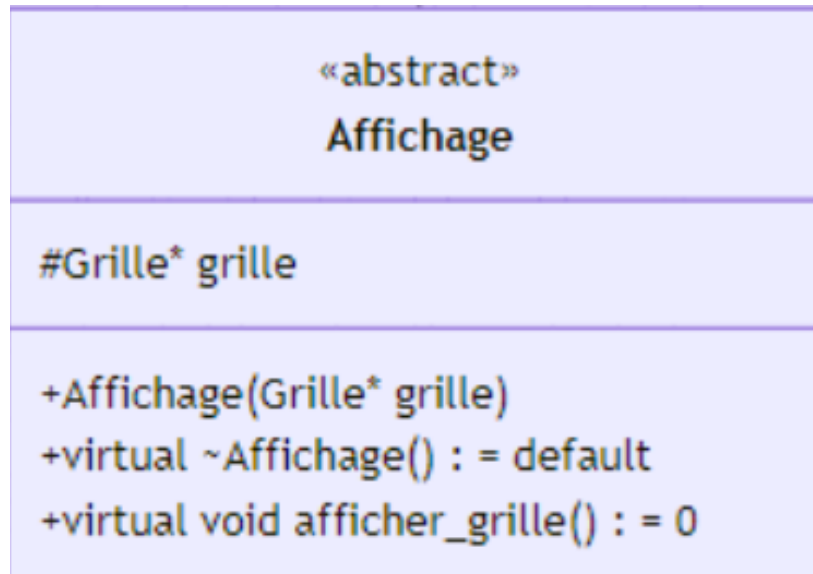
Efficacité : les méthodes parcourent la grille une seule fois pour calculer les statistiques, ce qui minimise la complexité.

Modularité : la classe **Compteurs** est conçue pour être utilisée avec n'importe quelle instance de **Grille**, ce qui la rend réutilisable dans d'autres simulations ou jeux.

Simplicité : les méthodes sont simples à utiliser et permettent une récupération rapide des informations clés sur la grille.

La classe Affichage

La classe **Affichage** est une classe abstraite conçue pour représenter la structure générale d'un système d'affichage de la grille. Elle fournit une interface pour afficher les données d'une instance de **Grille**, tout en laissant la mise en œuvre des détails spécifiques aux classes dérivées.



La classe **Affichage** est donc une classe abstraite servant de base à d'autres classes spécifique d'affichage.

Le seul et unique attribut protégé de cette classe est :

Grille* grille;

C'est un pointeur vers une instance de **Grille**. Cela permet à toutes les sous-classes d'accéder aux données de la grille.

Les méthode publiques sont les suivantes :

Constructeur :

Affichage(Grille* grille);

Associe une instance de **Grille** à la classe d'affichage.

Paramètre : grille, un pointeur vers une grille existante.

Destructeur virtuel :

virtual ~Affichage();

Marqué comme virtuel pour permettre une destruction correcte dans les classe sous-classes dérivées.

Méthode virtuelle pure :

virtual void afficher_grille() = 0;

Méthode abstraite que toutes les classes dérivées doivent implémenter. L'objectif étant d'afficher la grille de plusieurs manières.

Utilisation

Points importants :

Classe abstraite : la méthode **virtuelle pure afficher_grille()** rend **Affichage** non instanciable. Cela oblige les classes dérivées à définir leur propre logique d'affichage.

Flexibilité :

Affichage peut être adaptée pour différents types d'affichage en héritant et en personnalisant la méthode **afficher_grille()**.

Protection des données :

L'attribut grille est en mode **protected**, garantissant son accessibilité aux classes dérivées uniquement.

Polymorphisme : les objets **Affichage** peuvent être manipulés via des pointeurs ou référence vers la classe de base, tout en utilisant les implémentations spécifiques des classes dérivées.

Avantage :

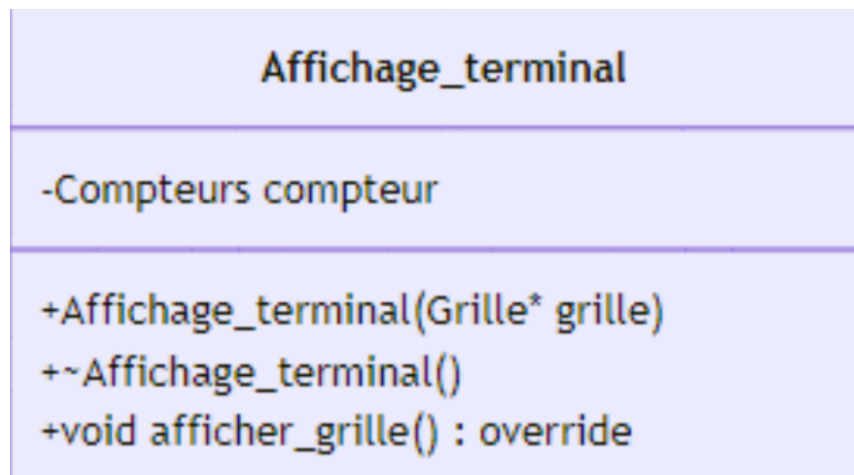
Extensibilité : de nouvelles méthodes d'affichage peuvent être ajoutées simplement en créant de nouvelles classes dérivées.

Réutilisabilité : la logique d'accès à la grille est centralisée dans la classes de base, évitant la duplication dans les classes dérivées.

Modularité : permet de séparer la logique d'affichage des autres aspects du programme.

La classe **Affichage_terminal**

La classe **Affichage_terminal** hérite de la classe abstraite **Affichage** et implémente une méthode concrète pour afficher une grille sur le terminal. Elle intègre également des informations supplémentaires sur la grille à l'aide de la classe **Compteurs**.



Elle est divisée en deux fichiers :

Affichage_terminal.h : qui contient la définition de la classes avec les attributs et les méthodes.

Affichage_terminal.cpp : qui contient l'implémentation des méthodes définies précédemment.

Cette classe hérite donc publiquement de **Affichage** et spécialise son comportement pour affichage dans le terminal.

L'attribut privé est le suivant :

Compteur compteur : une instance de la classe **Compteurs**, utilisée pour calculer des statistiques sur la grille, comme le nombre totales de cellules, les cellules vivantes, et les cellules mortes.

Constructeur :

Affichage_terminal(Grille* grille);

Initialise l'objet **Affichage_terminal** avec une grille qui prend en paramètre un pointeur vers une instance de la classe **Grille**.

Destructeur :

~Affichage_terminal();

Détruit l'instance **d'Affichage_terminal**. Ici, il ne fait rien mais il est inclus pour les bonnes pratiques en cas d'ajout de de ressources dynamiques.

La méthode publique est la suivante :

void afficher_grille() override;

Cette méthode implémente l'affichage de la grille dans un format lisible sur le terminal.

Elle affiche également des statistiques calculées par l'objet **Compteurs** :

Nombre total de cellules, nombre de cellules vivantes et nombre de cellules mortes.

Utilisation

Affichage d'une grille dans le terminal :

```
Grille ma_grille(5, 5); // Une grille de 5x5
Affichage_terminal affichage(&ma_grille);
affichage.afficher_grille();
```

Détails d'implémentation :

Affichage des cellules : la méthode parcourt chaque cellule de la matrice et affiche son statut.

Calcul des métriques : utilise les méthodes de la classe **Compteurs** pour afficher les statistiques de la grille.

Organisation visuelle : ajoute des espaces entre les statuts des cellules et un saut de ligne entre chaque ligne de la grille pour une meilleure visibilité.

Points importants :

Extension de la classe **Affichage** : l'utilisation de l'héritage permet de centraliser la logique d'accès à la grille tout en spécifiant l'affichage pour le terminal.

Statistiques intégrées : la classe **Compteurs** enrichit l'affichage en fournissant des informations clés sur l'état actuel de la grille.

Flexibilité future : si une nouvelle méthode de calcul des statistiques ou d'affichage est nécessaire, elle peut être ajoutée ou remplacée sans modifier la structure de la classe.

Avantages : clarté : permet aux utilisateurs de visualiser facilement l'état actuel de la grille dans le terminal.

Informativité : les statistiques supplémentaires offrent un aperçu global de l'état de la simulation.

Réutilisabilité : la classe peut être utilisée dans diverses simulations où un affichage terminal est requis.

La classe Affichage_graphique

La classe **Affichage_graphique** hérite de la classe abstraite **Affichage** et implémente une méthode concrète pour afficher la grille du jeu de la vie graphiquement à l'aide la bibliothèque SFML. Elle permet aussi d'afficher des informations supplémentaires concernant la grille à travers la classe **Compteurs**.

Affichage_graphique
<pre>-int cellule_taille -long int temps_entre_generation -Grille_update* grille_update -int window_colonne -int window_ligne -Compteurs compteur -sf::RenderWindow* window -sf::RenderWindow* windowCompteurs -int generation +Affichage_graphique(Grille* grille, long int temps_entre_generation) +~Affichage_graphique() +void afficher_grille() : override +void afficher_cellule() +void afficher_compteurs()</pre>

Elle est divisée en deux fichiers :

Affichage_graphique.h : qui contient la définition de la classes avec les attributs et les méthodes.

Affichage_graphique.cpp : qui contient l'implémentation des méthodes définies précédemment.

Les attributs privés de la classe sont les suivants :

Compteurs compteur : instance de la classe **Compteurs**, utilisée pour calculer des statistiques sur la grille, comme le nombre total de cellules, le nombre de cellules vivantes et mortes.

Grille_update* grille_update : pointeur vers un objet de type **Grille_update** qui gère la mise à jour de la grille en fonction de son mode.

Sf::RenderWindow* window : pointeur vers la fenêtre principale d'affichage

Sf::RenderWindow windowCompteurs : pointeur vers la fenêtre d'affichage des compteurs.

Int cellule_taille : taille de chaque cellule dans l'affichage graphique (en pixels).

Long int temps_entre_generation : temps en millisecondes entre chaque générations.

Int window_colonne : largeur de la fenêtre d'affichage (en pixel).

Int window_ligne : Hauteur de la fenêtre d'affichage (en pixels).

Int generation : compteur pour le nombre de génération dans le jeu.

Constructeur :

Affichage_graphique(Grille* grille, long int temps_entre_generation);

Initialise l'objet **Affichage_graphique** avec une grille donnée et un temps entre chaque génération. Il permet également de choisir si la grille est torique ou pas en créant une instance appropriée de la classe **Grille_update**. Il prend en paramètre **Grille* grille**, un pointeur vers une instance de la classe **Grille** qui représente la grille du jeu et **long int temps_entre_generation**, le temps entre chaque nouvelle génération.

Destructeur :

~Affichage_graphique();

Détruit l'instance d'**Affichage_graphique**.

La classe Affichage_graphique

Les méthodes publiques sont les suivantes :

void afficher_grille() override;

Cette méthode est responsable de l'affichage de la grille dans une fenêtre graphique créer avec SFML. Elle gère également l'affichage des compteurs dans une fenêtre séparée.

Détails de l'implémentation :

Crée deux fenêtre avec SFML : une pour afficher la grille des cellules et une autre pour afficher les compteurs des cellules vivantes, mortes et le nombre de générations.

Parcourt la grille et affiche chaque cellule sous forme de rectangle, avec une couleur différente pour les cellules vivantes (noir), mortes (blanc) et état spécifique (rouge).

Affiche les statistiques de la grille.

Met à jour la grille après chaque génération selon le type de mise à jour choisi.

Effectue une pause entre chaque génération selon le temps défini par **temps_entre_generation**.

void afficher_cellule();

void afficher_compteurs();

Les deux méthodes précédentes permettent de découper notre code en plusieurs méthodes afin de le rendre plus lisible. Elle gère respectivement la mise à jour des couleurs de des cellules et l'affichage des compteurs. Elles sont appelées dans la méthode **afficher_graphique**.

Utilisation

Affichage d'une grille sur une fenêtre gérée par SFML

```
Grille ma_grille(5, 5); // Créer une grille 5x5
Affichage_graphique affichage(&ma_grille, 500); // Créer un affichage
graphique avec un délai de 500 ms par génération
affichage.afficher_grille(); // Afficher la grille et les compteurs
```

Points importants :

Extension de la classe **Affichage** : en héritant de **Affichage**, cette classes centralise la gestion de l'accès à la grille tout en spécialisant l'affichage graphique via SFML.

Statistiques intégrées : Grâce à l'utilisation de la classe **Compteurs**, des informations clés sur l'état de la grille sont affichées en temps réel.

Flexibilité future : si de nouvelles méthodes d'affichage ou de calcul des statistiques sont nécessaires, elles peuvent facilement ajoutées ou remplacées sans modifier la structure de la classe.

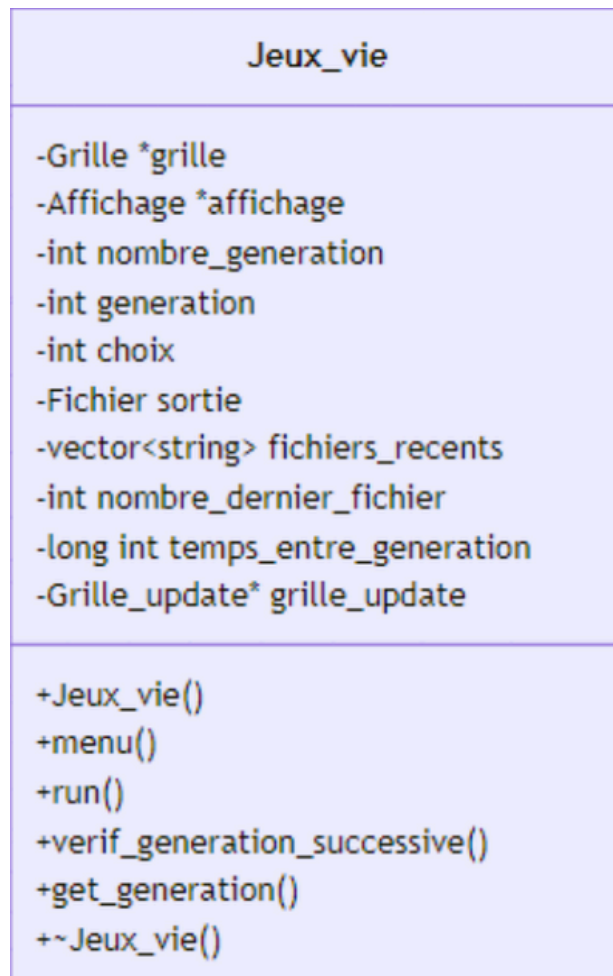
Avantages : Clarté : permet une visualisation graphique claire de l'état de la grille, facilitant la compréhension de l'évolution du jeu de la vie.

Informativité : les statistiques supplémentaires apportent un aperçu détaillé de la simulation.

Réutilisabilité : cette classe peut être utilisée dans d'autres simulations nécessitant un affichage graphique de type grille.

La classe Jeux_vie

Cette classe représente le jeu de la vie, avec des fonctionnalités pour afficher l'état de la grille, gérer les générations successives et vérifier la répétitions des générations. Elle permet également de gérer les fichiers relatifs aux états de la grille au cours de l'exécution.



Elle est divisée en deux fichiers :

Jeux_vie.h : qui contient la définition de la classes avec les attributs et les méthodes.

Jeux_vie.cpp : qui contient l'implémentation des méthodes définies précédemment.

En voici les attributs privés :

Grille* grille : Pointeur vers une instance de la classe **Grille**, représentant la matrice de cellules.

Affichage* affichage : Pointeur vers une instance de la classe **Affichage**, utilisée pour afficher la grille (soit en terminal, soit graphiquement).

Int nombre_generation : Nombre de générations à exécuter.

Int generation : Compteur pour suivre l'index de la génération actuelle.

Int choix : Choix de l'utilisateur pour le mode d'affichage.

Fichier sortie : Instance de la classe **Fichier** pour la gestion des fichiers de sortie.

Std::vector<std::string> fichiers_recents : Vecteur pour gérer les derniers fichiers créés.

Int nombre_dernier_fichier : Nombre de fichiers récents à stocker pour la vérification des répétitions de générations.

Long int temps_entre_generation : Temps en millisecondes entre chaque génération pour l'affichage graphique.

Grille_update* grille_update : Pointeur vers une instance de **Grille_update**, définissant la manière dont la grille est mise à jour (par défaut ou torique).

La classe Jeux_vie

Les méthodes publiques :

Jeux_vie();

Constructeur de la classe qui initialise le jeu en fonction du choix de l'utilisateur.

Si l'utilisateur choisit l'affichage terminal une instance de **Affichage_terminal** est créée.

Si l'utilisateur choisit l'affichage graphique une instance de **Affichage_graphique** est créée.

Selon le choix, une grille torique ou non torique est configurée.

Le nombre de générations et d'autres paramètres sont configurés.

void menu();

Affiche le menu principal du jeu et permet à l'utilisateur de choisir l'option d'affichage ou de quitter.

void run();

Lance l'exécution du jeu, effectuant les mises à jour de la grille pour chaque génération

Vérifie si les générations successives sont identiques à l'aide de

verif_generation_successive().

Affiche la grille et écrit l'état actuel dans un fichier.

Mette à jour la grille en fonction du mode sélectionné (torique ou non torique).

void verif_generation_successive();

Elle vérifie si deux générations successives sont identiques en comparant les fichiers générés.

Si elles sont identiques le programme s'arrête.

int get_generation(); : elle retourne le nombre de génération.

~Jeux_vie();

Destructeur de la classe, libère la mémoire allouée pour grille et affichage.

Utilisation

Lancé le jeu :

```
Jeux_vie jeux_vie;  
jeux_vie.run();
```

Points importants :

Vérification des générations successives : la vérification des générations identiques est une fonctionnalité clé qui permet d'éviter les boucles infinies dans le jeu, en s'assurant que le programme s'arrête si les générations se répètent.

Affichage flexible : le jeu peut être exécuté avec un affichage en mode texte ou graphique, offrant ainsi une flexibilité d'usage.

Gestion des fichiers : les fichiers de génération sont utilisés pour stocker l'état de la grille, ce qui permet de vérifier les répétitions entre générations.

Avantages :

Modularité : le jeu peut facilement être adapté pour différents types d'affichages et modes de mise à jour de la grille.

Simplicité : l'interface utilisateur est simple, avec des choix clairs pour l'affichage et la gestion des générations.

Efficacité : le programme vérifie efficacement les répétitions de générations, assurant ainsi que le jeu s'arrête lorsqu'une boucle infinie de générations se forme.

TESTS UNITAIRES

Le fichier test.h

Le fichier **test.h** contient des fonctions de test pour vérifier le bon fonctionnement de différentes fonctionnalités du programme, principalement liées à la gestion de fichiers, à la manipulation de grilles (matrices), et à la gestion des cellules. Chaque fonction effectue des tests unitaires pour valider des aspects spécifiques du programme, en utilisant des assertions pour vérifier que les résultats attendus correspondent aux résultats obtenus.

Vous trouverez ci-après un aperçu de la fonction globale de chaque méthode dans le fichier test.h :

test_lecture_fichier : Teste la lecture d'un fichier et la validation de la matrice lue, en vérifiant les dimensions du fichier et certains éléments de la matrice. Elle s'assure que la lecture du fichier fonctionne comme prévu.

test_ecriture_fichier : Vérifie l'écriture d'une matrice dans un fichier. Elle teste si le fichier est correctement créé et si le contenu de la matrice est correctement écrit, en validant les dimensions et les valeurs des cellules.

test_creer_dossier : Teste la création d'un dossier en utilisant la méthode **creer_dossier()**. Elle vérifie si le dossier est effectivement créé à l'emplacement attendu.

test_supprimer_dossier : Vérifie que la méthode **supprimer_dossier()** fonctionne correctement en supprimant un dossier, puis vérifie que le dossier n'existe plus après la suppression.

test_get_nom : Teste la récupération du nom d'un fichier en s'assurant que le nom du fichier est correctement renvoyé par la méthode **get_nom()**.

test_grille_creation : Teste la création d'une grille et vérifie les dimensions et les valeurs des cellules dans la matrice créée.

test_set_ligne : Vérifie la méthode **set_ligne()** pour modifier le nombre de lignes d'une grille, en s'assurant que la modification est bien prise en compte.

test_set_colonne : Vérifie la méthode **set_colonne()** pour modifier le nombre de colonnes d'une grille, en s'assurant que la modification est bien prise en compte.

test_set_matrice : Teste la modification d'une matrice dans une grille et vérifie que la nouvelle matrice a bien été appliquée.

test_get_matrice : Vérifie la récupération de la matrice d'une grille en s'assurant que la méthode **get_matrice()** renvoie correctement la matrice et les valeurs attendues.

test_cellule_creation_par_defaut : Teste la création d'une cellule avec des valeurs par défaut et vérifie les valeurs initiales.

test_cellule_creation_par_parametres : Vérifie la création d'une cellule avec des paramètres spécifiques et s'assure que les valeurs initiales sont correctes.

test_getters : Teste les méthodes getter pour récupérer les informations d'une cellule (coordonnées et statut).

Le fichier test.h

test_status_update : Vérifie la mise à jour du statut d'une cellule, en s'assurant que la méthode **status_update()** fonctionne correctement pour modifier le statut de la cellule.

test_affichage_terminal_et_compteurs : Teste l'affichage dans le terminal et la gestion des compteurs en créant une grille avec un motif spécifique, puis en la configurant et en affichant son état. Ces tests permettent de vérifier que les différentes fonctionnalités du programme, notamment celles liées à la gestion de fichiers et de grilles, fonctionnent correctement en simulant des cas d'utilisation réalistes. Les assertions sont utilisées pour s'assurer que les valeurs obtenues pendant l'exécution correspondent aux résultats attendus, garantissant ainsi la fiabilité du code.

test_grille_update_default : Cette fonction teste la mise à jour d'une grille classique avec un motif spécifique. Elle crée une grille de 5x5, initialise certaines cellules comme vivantes, puis applique la mise à jour par défaut (probablement basée sur un certain nombre de règles de voisinage, comme celles utilisées dans le jeu de la vie). Ensuite, elle vérifie si l'état des cellules correspond aux attentes après la mise à jour.

test_grille_update_torique : Ce test vérifie la mise à jour d'une grille dans un environnement torique (où les bords de la grille sont connectés entre eux, comme dans le cas de l'effet "tore" dans certaines simulations). La grille est lue à partir d'un fichier texte et une mise à jour est effectuée selon les règles du voisinage torique. Ensuite, elle vérifie que la grille a bien été mise à jour en fonction des règles définies pour les voisins en torique.

Une fonction **test()** a aussi été ajoutée pour faciliter l'appel des fonctions dans le **main.cpp**

```
void test (){\n    test_lecture_fichier();\n    test_ecriture_fichier();\n    test_creer_dossier();\n    test_supprimer_dossier();\n    test_get_nom();\n    test_grille_creation();\n    test_set_ligne();\n    test_set_colonne();\n    test_set_matrice();\n    test_get_matrice();\n    test_cellule_creation_par_defaut();\n    test_cellule_creation_par_parametres();\n    test_getters();\n    test_status_update();\n    test_affichage_terminal_et_compteurs();\n    test_affichage_graphique_et_compteurs();\n    test_grille_update_default();\n    test_grille_update_torique();\n}
```

Le fichier main.cpp

Le fichier main.cpp contient le point d'entrée principal du programme, où l'objet **Jeux_vie** est créé et exécuté. Ce fichier permet à la fois de lancer le Jeu de la Vie, mais aussi de lancer une campagne de test unitaire afin de vérifier si le programme fonctionne correctement.

La fonction principale est le point d'entrée du programme. Elle propose deux choix:

1. Faire des tests unitaires
2. Lancer le Jeu de la Vie

Si l'utilisateur entre "1" alors il passe en phase de test. Les résultats des tests unitaires seront affichés dans la console.

Si l'utilisateur entre "0" alors la Jeu de la Vie se lance et l'utilisateur peut choisir son interface, son fichier d'entrée, son nombre de génération pour la partie terminal et ainsi de suite.

Avantages :

Simplicité : Le fichier est simple et direct, il demande d'abord un choix à l'utilisateur, puis agit en fonction de ce choix, soit en lançant le jeu de la vie, soit en exécutant les tests unitaires.

Modularité : La logique du jeu de la vie est encapsulée dans la classe **Jeux_vie**, ce qui permet de maintenir une séparation claire entre la logique du programme et son interface utilisateur.

Facilité de test : L'intégration de tests unitaires via le fichier test.h permet de tester indépendamment les différentes parties du programme sans interférer avec l'exécution normale du jeu.

Point à retenir :

Interaction avec l'utilisateur : L'utilisateur choisit entre les tests unitaires ou l'exécution du jeu via une interface simple en ligne de commande.

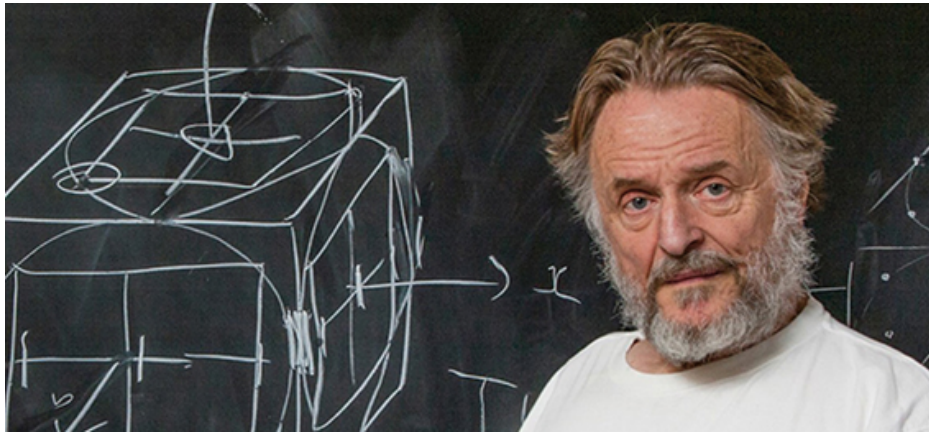
Exécution centralisée : Le fichier main.cpp gère l'exécution du programme, de manière à rendre l'exécution claire et structurée.

PARTIE UTILISATEUR

PRÉSENTATION DU JEU DE LA VIE

Jeu de la vie :

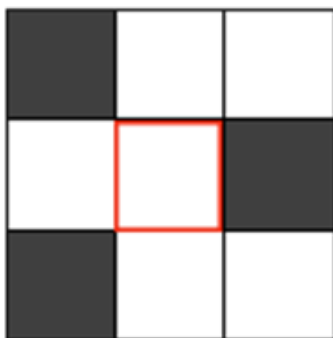
Défini dans les années 70 par le mathématicien américain John Horton Conway, le jeu de la vie appartient à une vaste classe d'algorithmes appelés automates cellulaires. Ces systèmes ont pour but de simuler l'évolution d'une population de cellules vivantes réparties sur une grille. Comme pour les fractales, on peut obtenir des comportements très complexes avec des règles d'évolution très simples. Malgré son ancienneté, le jeu de la vie fait toujours l'objet d'intenses recherches, avec souvent des résultats surprenants : on a ainsi pu montrer qu'il se comporte comme un ordinateur universel, capable notamment de calculer (et d'écrire) les décimales de π !



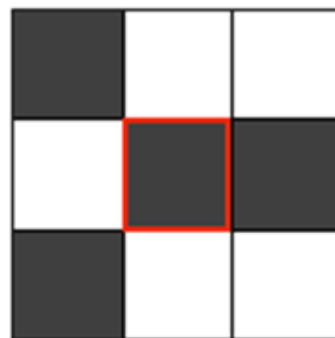
Explication des règles de jeu de la vie :

La règle de transition locale se décline en deux sous règles, qui dépendent du nombre de voisins vivants parmi les huit cellules adjacentes (Nord, Sud, Est, Ouest ainsi que les cellules diagonales) :

- (A) La règle de naissance stipule qu'une cellule passe de l'état mort à l'état vivant si elle est entourée de trois cellules vivantes.



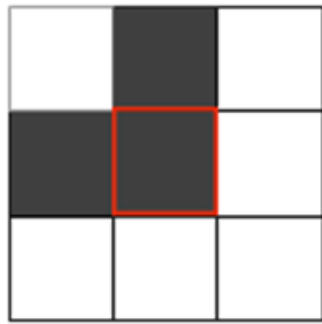
T



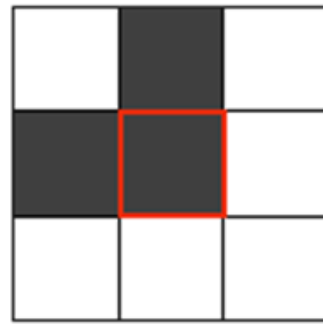
T+1

La cellule au centre de la grille (en blanc cerclée de rouge) passe de l'état mort (blanc) à celui de vivant (gris) au temps T+1 car elle est entourée de trois cellules vivantes.

- (B) La règle de naissance stipule qu'une cellule passe de l'état mort à l'état vivant si elle est entourée de trois cellules vivantes.



T



T+1

La cellule grise au centre de la grille est vivante au temps T et elle le restera au temps T+1 car elle est toujours entourée de deux cellules vivantes.

Structures :

Structures stables :

Les structures stables (en anglais still life) sont des ensembles de cellules ayant stoppé toute évolution : elles sont dans un état stationnaire et n'évoluent plus tant qu'aucun élément perturbateur n'apparaît dans leur voisinage. Un bloc de quatre cellules est la plus petite structure stable possible.

Certaines figures se stabilisent en structures florales après une succession d'itérations comparables à une floraison.



Vaisseaux :

Les vaisseaux — ou navires — (en anglais spaceships, « vaisseaux spatiaux ») sont des structures capables, après un certain nombre de générations, de produire une copie d'elles-mêmes, mais décalées dans l'univers du jeu.



OBTENTION DU CODE

Prérequis :

1. Système d'exploitation :

- Vous devez disposer d'un environnement Linux. Vous pouvez utiliser WSL (Windows Subsystem for Linux) sur Windows ou une machine virtuelle avec une distribution Linux (comme Ubuntu, Debian, etc.).

2. Compilateur Clang++ et Make :

- Assurez-vous que le compilateur Clang++ et l'outil make sont installés sur votre environnement Linux.



Étapes d'installation:

1. Installez WSL (si vous utilisez Windows) :

- Suivez les instructions sur le site officiel de Microsoft pour installer WSL : [Guide d'installation de WSL](#).

2. Installer une distribution Linux :

- Une fois que WSL est installé, installez une distribution Linux comme Ubuntu via le Microsoft Store.
- Ouvrez le terminal WSL et configurez votre distribution Linux.

3. Installer Clang++ et Make :

- Ouvrez votre terminal Linux (WSL ou machine virtuelle) et exécutez les commandes suivantes pour mettre à jour les paquets et installer Clang++ et Make :

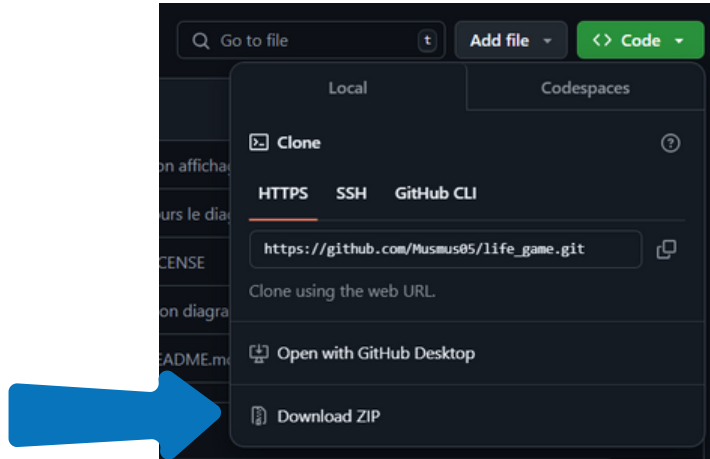
```
~$ sudo apt update  
~$ sudo apt install clang make
```

Comment obtenir le jeu (le code) :

Pour obtenir le code, vous avez 2 possibilités :

1. Télécharger le code en format ZIP :

- Rendez-vous sur la page principale du dépôt [Musmus05/life_game](https://github.com/Musmus05/life_game).
- Cliquez sur le bouton vert "Code".
- Sélectionnez "Download ZIP" pour télécharger le code sous forme de fichier ZIP.
- Une fois téléchargé, décompressez le fichier pour accéder aux fichiers du projet.



2. Cloner le dépôt Git :

- Ouvrez votre terminal.
- Exécutez la commande suivante pour cloner le dépôt sur votre machine locale :

```
~$ git clone https://github.com/Musmus05/life_game.git
```

- Accédez au répertoire du projet cloné :
- `cd life_game`

```
~$ cd life_game
```

Comment obtenir les dernières versions :

Pour obtenir les dernières versions du code, veuillez suivre les étapes :

1. Télécharger le code en format ZIP :

- Rendez-vous sur la page principale du dépôt [Musmus05/life_game](https://github.com/Musmus05/life_game).
- Cliquez sur [Releases](#).
- Sélectionnez "Source code ZIP" pour télécharger la dernière version du code sous forme de fichier ZIP.



Remarque : Si vous avez Cloner le dépôt du projet auparavant il faut juste changer de branche en V.2 :

V.2 Tag at 4a292c8b

tags

LANCEMENT DU JEU

Lancer le jeu :

Compiler le projet :

1. Accédez au répertoire src :

```
/life_game$ cd src/
```

2. Utilisez la commande make pour compiler le projet :

```
/life_game/src$ make
```

Le résultat :

```
clang++ -lsfml-graphics -lsfml-window -lsfml-audio -lsfml-network -lsfml-system -g -Wmost -Werror ./compteurs.cpp ./Affichage_terminal.cpp ./Affichage_graphique.cpp ./Jeux_vie.cpp ./Grille_update_default.cpp ./Fichier.cpp ./main.cpp ./Cellule.cpp ./Grille.cpp ./Grille_update_torique.cpp -o "main"
```

3. Exécuter le projet : Une fois la compilation terminée, exécutez le jeu avec la commande suivante :

```
/life_game/src$ ./main
```

Les résultats attendus:

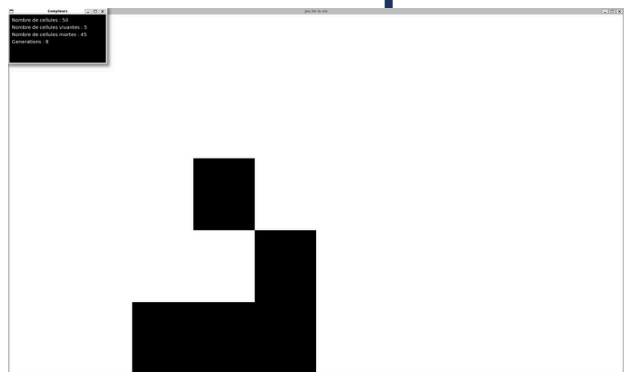
Version 1 :

- Après l'exécution du jeu, vous arrivez au menu du choix de l'affichage, c'est là où vous choisissez entre une interface terminale ou une interface graphique ou de quitter.

```
Generation : 0
0 1 0 0 0 0 0 0 0 0
0 0 1 0 0 0 0 0 0 0
1 1 1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0

Nombre de cellules : 50
Nombre de cellules vivantes : 5
Nombre de cellules mortes : 45
=====
Generation : 1
0 0 0 0 0 0 0 0 0 0
1 0 1 0 0 0 0 0 0 0
0 1 1 0 0 0 0 0 0 0
0 1 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0

Nombre de cellules : 50
Nombre de cellules vivantes : 5
Nombre de cellules mortes : 45
=====
```



Les résultats attendus:

Version 1 :

- Si votre choix est celui du terminal en mettant 1, vous devez mettre le nom du fichier source là où vous pouvez mettre le nombre de lignes et de colonnes que vous souhaitez avoir et proposer des cellules mortes avec des 0 ou en vie avec des 1.

Nom du fichier source ici c'est data.txt

Lignes

colonnes

cellules morte

cellule en vie

```
src > ≡ data.txt
1 5 10
2 0 1 0 0 0 0 0 0 0
3 0 0 1 0 0 0 0 0 0
4 1 1 1 0 0 0 0 0 0
5 0 0 0 0 0 0 0 0 0
6 0 0 0 0 0 0 0 0 0
```

- Pour l'instant vous devez avoir ceci :

```
Setting vertical sync not supported
musmus@PC:~/livrable/life_game/src$ ./main
1. Affichage terminal
2. Affichage graphique
3. Quitter
Choisissez une option : 1
Nom du fichier : data.txt
```

- Ensuite vous devez mettre le nombre de génération souhaité, c'est-à-dire nombre de fois que les règles du jeu sont appliquées pour faire évoluer l'état de la grille
- Puis, combien de fichiers voulez-vous examiner pour vérifier la répétition des dernières générations? (Si cela arrive, le programme s'arrêtera et ne continuera pas car il n'y a aucun intérêt de visualiser des générations qui n'évoluent pas). Par exemple, vous décidez de choisir une vérification de 3 fichiers, c'est-à-dire que le jeu va vérifier à chaque 3 générations successives si la grille n'évolue pas. Si c'est le cas, le programme s'arrête en affichant "Il y a 3 générations identiques. Arrêt du programme." Si ce n'est pas le cas, l'affichage continue jusqu'à il y a une répétition pendant 3 générations ou à la fin du nombre de génération demandée.

Les résultats attendus:

Version 1 :

- Ensuite vous devez choisir entre une évolution torique ou non torique :

Évolution non torique :

1. Bords fixes : Les cellules sur les bords de la grille ont moins de voisins que celles au centre, ce qui peut affecter leur évolution.
2. Effet de bord : Les cellules sur les bords peuvent mourir plus facilement ou ne pas se reproduire correctement en raison du manque de voisins.

Évolution torique :

1. Grille enroulée : La grille est considérée comme enroulée sur elle-même, comme la surface d'un tore (donut).
2. Voisins continus : Les cellules sur les bords de la grille ont des voisins de l'autre côté de la grille. Par exemple, une cellule sur le bord gauche a pour voisin une cellule sur le bord droit.
3. Homogénéité : Cela permet une évolution plus homogène, sans effets de bord, car chaque cellule a toujours le même nombre de voisins potentiels.

Exemple visuel :

Imaginez une grille 3x3 :

1 2 3
4 5 6
7 8 9

- Non torique : La cellule 1 n'a que trois voisins (2, 4, 5).
- Torique : La cellule 1 a huit voisins (2, 3, 4, 5, 6, 7, 8, 9), car elle est connectée aux cellules de l'autre côté de la grille.

Les résultats attendus:

Version 1 :

- Affichage terminal :

```
1. Affichage terminal
2. Affichage graphique
3. Quitter
Choisissez une option : 1
Nom du fichier : data.txt
Nombre de génération : 10
Combien de fichiers voulez-vous examiner pour vérifier la répétition des dernières générations : 2
torique ou non torique ? (1/0) : 1
Generation : 0
0 1 0 0 0 0 0 0 0 0
0 0 1 0 0 0 0 0 0 0
1 1 1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
Nombre de cellules : 50
Nombre de cellules vivantes : 5
Nombre de cellules mortes : 45
=====
Generation : 1
0 0 0 0 0 0 0 0 0 0
1 0 1 0 0 0 0 0 0 0
0 1 1 0 0 0 0 0 0 0
0 1 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
Nombre de cellules : 50
Nombre de cellules vivantes : 5
Nombre de cellules mortes : 45
=====
Generation : 2
0 0 0 0 0 0 0 0 0 0
0 0 1 0 0 0 0 0 0 0
1 0 1 0 0 0 0 0 0 0
0 1 1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
Nombre de cellules : 50
Nombre de cellules vivantes : 5
Nombre de cellules mortes : 45
=====
```

la n-ème
génération ici
c'est 0

Les compteurs

la n-ème
génération ici c'est 1

L'état de la grille de la generation 0 C'est votre fichier initiale

L'état de la grille de la génération 1. Les règles du jeu de la vie appliquées à la génération 0

L'état de la grille de la génération 2. Les règles du jeu de la vie appliquées à la génération 1

Les résultats attendus:

Version 1 :

- La sauvegarde des générations dans dossier Sortie_sauvegarde :

Chaque fichier de sortie contient les dimensions de la grille (nombre de lignes et de colonnes) sur la première ligne.

Les lignes suivantes contiennent l'état de chaque cellule de la grille (0 pour morte, 1 pour vivante, 2 pour un état spécial).

Nombre de fichiers de sortie :

- Si vous choisissez de simuler 10 générations, il y aura 11 fichiers de sortie (de data_out_0.txt à data_out_10.txt), car la génération initiale (génération 0) est également sauvegardée.



Prenons un exemple, si on prend le fichier sauvegarde de la 2ème génération, donc il y aura les mêmes données que l'affichage

```
Sortie_sauvegarde > data_out_2.txt
5 10
0 0 0 0 0 0 0 0 0 0
0 0 1 0 0 0 0 0 0 0
1 0 1 0 0 0 0 0 0 0
0 1 1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
```

```
Generation : 2
0 0 0 0 0 0 0 0 0 0
0 0 1 0 0 0 0 0 0 0
1 0 1 0 0 0 0 0 0 0
0 1 1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
```

```
Nombre de cellules : 50
Nombre de cellules vivantes : 5
Nombre de cellules mortes : 45
```


Les résultats attendus:

Version 1 :

- Si votre choix est celui du graphique en mettant 2, vous devez mettre le nom du fichier source là où vous pouvez mettre le nombre de lignes et de colonnes que vous souhaitez avoir et proposer des cellules mortes avec des 0 ou en vie avec des 1.

Nom du fichier source ici c'est data.txt

Lignes

colonnes

cellules morte

cellule en vie

```
src > ≡ data.txt
1 5 10
2 0 1 0 0 0 0 0 0 0
3 0 0 1 0 0 0 0 0 0
4 1 1 1 0 0 0 0 0 0
5 0 0 0 0 0 0 0 0 0
6 0 0 0 0 0 0 0 0 0
```

- Pour l'instant vous devez avoir ceci :

```
musmus@PC:~/livrable/life_game/src$ ./main
1. Affichage terminal
2. Affichage graphique
3. Quitter
Choisissez une option : 2
Nom du fichier : data.txt
Combien de temps entre chaque génération voulez-vous (en milliseconde) : █
```

- Puis vous devez avoir la question "Combien de temps entre chaque génération voulez-vous (en millisecondes) ?". Cette question se réfère à l'intervalle de temps entre chaque mise à jour de la grille dans le jeu de la vie. Cet intervalle est souvent utilisé pour contrôler la vitesse à laquelle les générations sont calculées et affichées.

Intervalle entre les générations

- Intervalle en millisecondes : C'est le temps d'attente entre chaque génération. Par exemple, un intervalle de 1000 millisecondes (1 seconde) signifie que la grille sera mise à jour toutes les secondes.
- Calcul du Frame Rate : Si vous avez un intervalle de 1000 millisecondes entre chaque génération, cela correspond à un frame rate de 1 FPS (1 frame par seconde). Si l'intervalle est de 500 millisecondes, cela correspond à 2 FPS (2 frames par seconde).

Les résultats attendus:

Version 1 :

- Ensuite vous devez choisir entre une évolution torique ou non torique :

Évolution non torique :

1. Bords fixes : Les cellules sur les bords de la grille ont moins de voisins que celles au centre, ce qui peut affecter leur évolution.
2. Effet de bord : Les cellules sur les bords peuvent mourir plus facilement ou ne pas se reproduire correctement en raison du manque de voisins.

Évolution torique :

1. Grille enroulée : La grille est considérée comme enroulée sur elle-même, comme la surface d'un tore (donut).
2. Voisins continus : Les cellules sur les bords de la grille ont des voisins de l'autre côté de la grille. Par exemple, une cellule sur le bord gauche a pour voisin une cellule sur le bord droit.
3. Homogénéité : Cela permet une évolution plus homogène, sans effets de bord, car chaque cellule a toujours le même nombre de voisins potentiels.

Exemple visuel :

Imaginez une grille 3x3 :

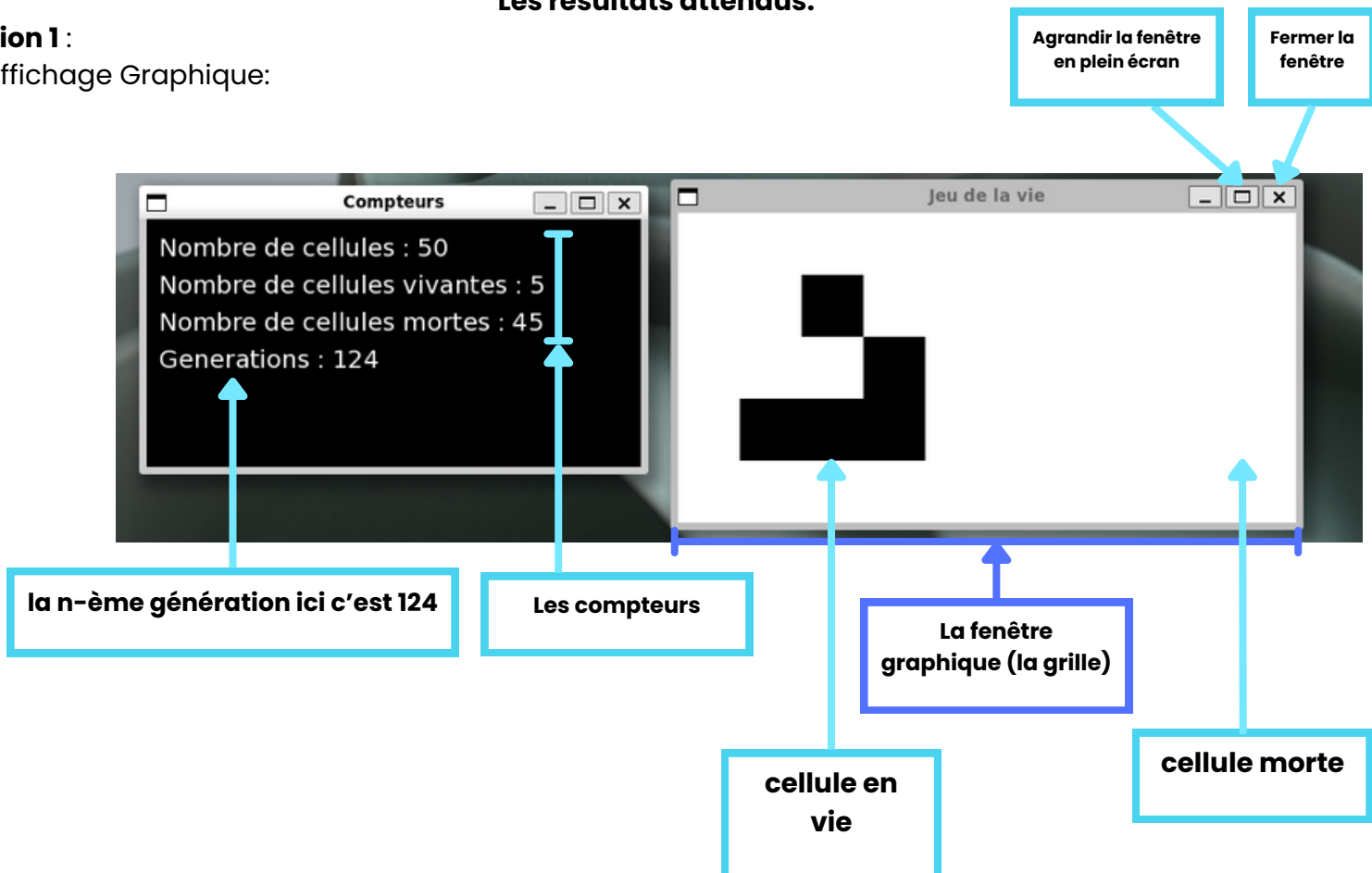
1	2	3
4	5	6
7	8	9

- Non torique : La cellule 1 n'a que trois voisins (2, 4, 5).
- Torique : La cellule 1 a huit voisins (2, 3, 4, 5, 6, 7, 8, 9), car elle est connectée aux cellules de l'autre côté de la grille.

Les résultats attendus:

Version 1 :

- Affichage Graphique:



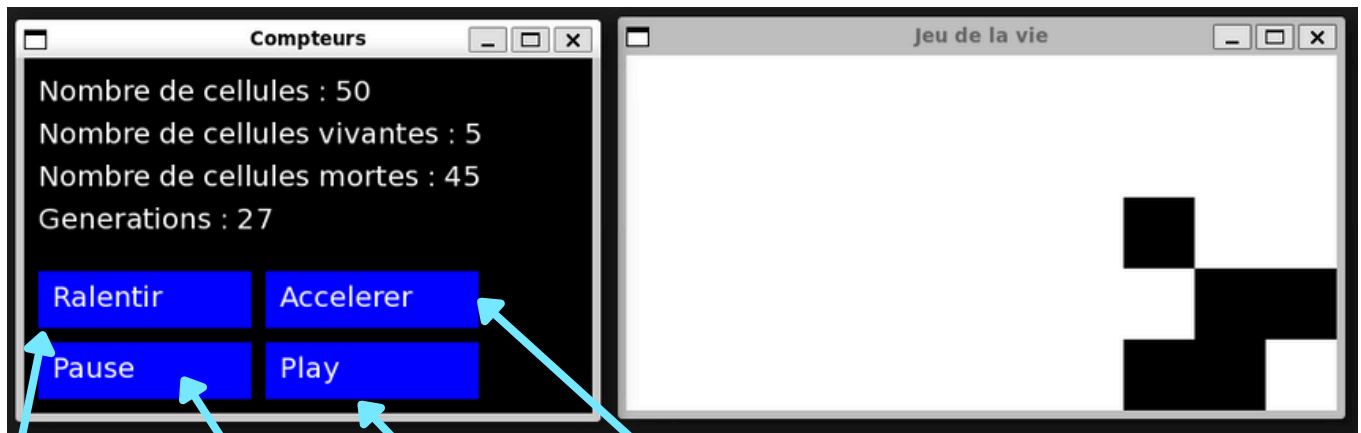
Les résultats attendus:

Version 2 :

- Dans la version 2 du projet, plusieurs améliorations ont été apportées par rapport à la version 1, notamment l'ajout de boutons dans l'affichage graphique.

Quatre boutons ont été ajoutés pour contrôler la simulation :

- Ralentir : Augmente le temps entre les générations.
 - Accélérer : Diminue le temps entre les générations.
 - Pause : Met la simulation en pause.
 - Play : Relance la simulation après une pause.
- La partie terminale reste la même que dans la version 1, Les méthodes et fonctionnalités de cette partie n'ont pas été modifiées.



**Bouton
Ralentir**

**Bouton
Pause**

**Bouton
Play**

**Bouton
Accélérer**

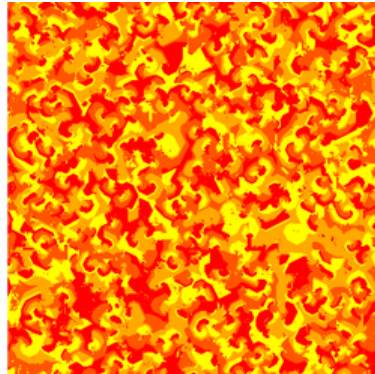
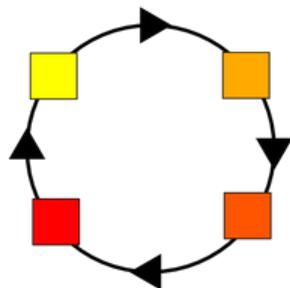
ANNEXES

AUTOMATE CELLULAIRE :

Un automate cellulaire est un ensemble de « cellules » regroupé en une seule grille, donc chacune contient un état choisi parmi un ensemble fini et qui peut évoluer au cours du temps. Le changement d'état d'une cellule est fait en fonction de son état actuel et du temps $t+1$ et de l'état des cellules adjacentes. A chaque nouvelle itération du temps, l'ensemble des état de toutes les cellules est actualisé. Cela créer donc une nouvelle génération.

Par exemple, le « Jeu de la vie » est un automate cellulaire à deux dimensions. Chaque cellule de sa grille peut avoir deux états : « vivante » ou « morte » aussi notée « 0 » ou « 1 ». Le prochain état de la cellule sera déterminé par l'état actuel de cette dernière ainsi que par le nombre de cellule vivante autour d'elle.

https://fr.wikipedia.org/wiki/Automate_cellulaire



MODE CONSOLE :

Le mode console représente l'affichage des données et du programme en général dans le terminal d'une machine. Dans notre cas, il servira à l'affichage des générations, du nombre de cellule morte/vivante. Il sert finalement d'interface entre l'homme et la machine.

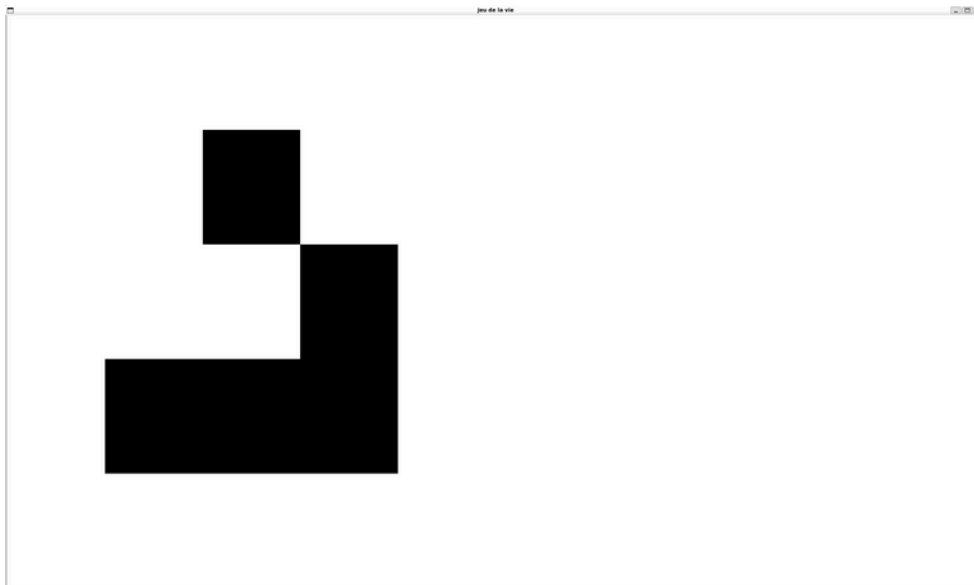
```
Generation : 0
0 1 0 0 0 0 0 0 0 0
0 0 1 0 0 0 0 0 0 0
1 1 1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0

Nombre de cellules : 50
Nombre de cellules vivantes : 5
Nombre de cellules mortes : 45
=====
Generation : 1
0 0 0 0 0 0 0 0 0 0
1 0 1 0 0 0 0 0 0 0
0 1 1 0 0 0 0 0 0 0
0 1 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0

Nombre de cellules : 50
Nombre de cellules vivantes : 5
Nombre de cellules mortes : 45
=====
```

MODE GRAPHIQUE :

Le mode graphique représente l'affichage des données et du programme en général dans une fenêtre programmé par le développeur du logiciel. Dans notre cas, cette fenêtre affichera les cycles de générations, le nombre de cellule, leur état. Cette fenêtre sert aussi d'interface homme-machine.

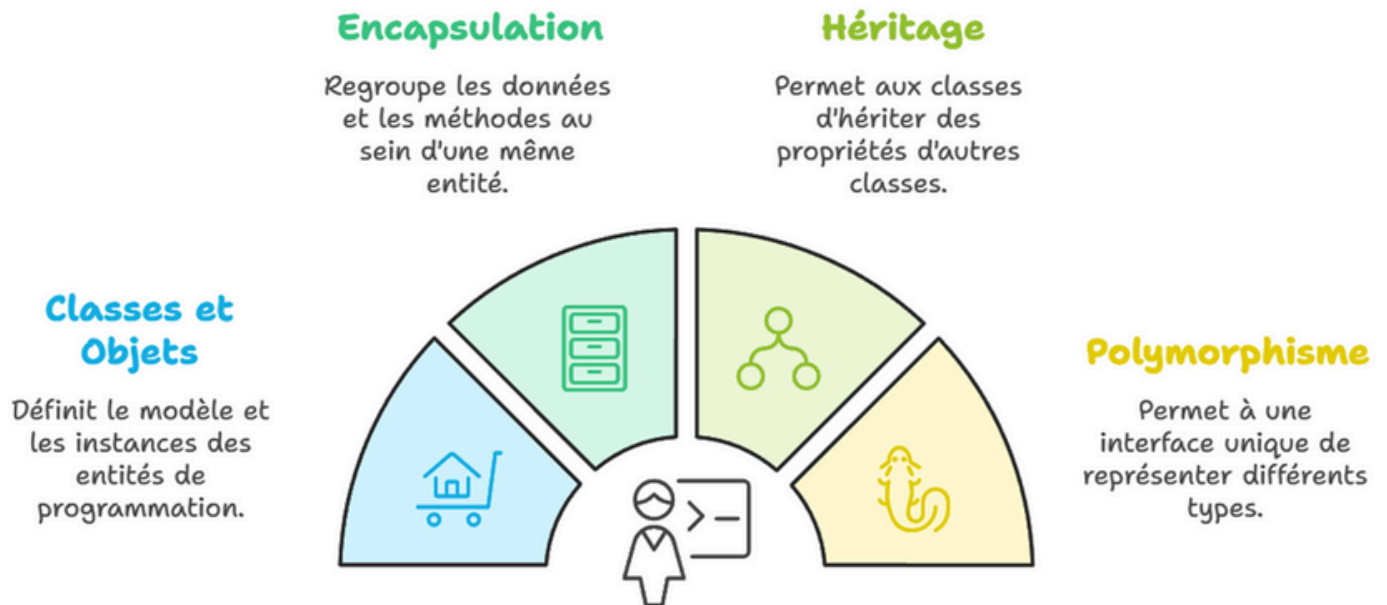


LA PROGRAMMATION ORIENTEE OBJET : La POO est un paradigme de programmation qui utilise des "objets" pour représenter des données et des méthodes. Bien que le C ne soit pas un langage orienté objet à proprement parler, il est possible d'implémenter certains concepts de la POO. En revanche, le C++ a été conçu pour supporter pleinement la POO. Ce cours abordera les concepts fondamentaux de la POO, les différences entre C et C++, ainsi que des exemples pratiques.

La programmation orientée objet repose sur plusieurs concepts clés :

1. **Classes et Objets :** Une classe est une structure qui définit un type d'objet. Un objet est une instance d'une classe.
2. **Encapsulation :** Cela consiste à regrouper les données et les méthodes qui opèrent sur ces données au sein d'une même entité (la classe).
3. **Héritage :** C'est la capacité d'une classe à hériter des propriétés et des méthodes d'une autre classe.
4. **Polymorphisme :** Cela permet d'utiliser une interface unique pour représenter des types différents.

Programmation Orientée Objet



CLASSE : Une classe est un modèle ou un plan qui définit les propriétés (attributs) et les comportements (méthodes) que les objets créés à partir de cette classe posséderont. En d'autres termes, une classe est une structure de données qui permet de regrouper des données et des fonctions qui opèrent sur ces données.

```
class Voiture {  
public:  
    // Attributs  
    string couleur;  
    string vitesse;  
  
    // Méthode  
    void rouler() {  
        cout << "La voiture roule." << endl;  
    }  
};
```

Nom de la classe →

Voiture

Attributs →

couleur
vitesse

Méthode →

rouler()

Dans cet exemple, Voiture est une classe avec deux attributs (couleur, vitesse) et une méthode (rouler). Les objets créés à partir de cette classe auront ces attributs et pourront exécuter ces méthodes.

OBJET : On dit qu'un objet est une instance d'une classe. Cela signifie qu'un objet est la matérialisation concrète d'une classe (tout comme la maison est la matérialisation concrète du plan de la maison).

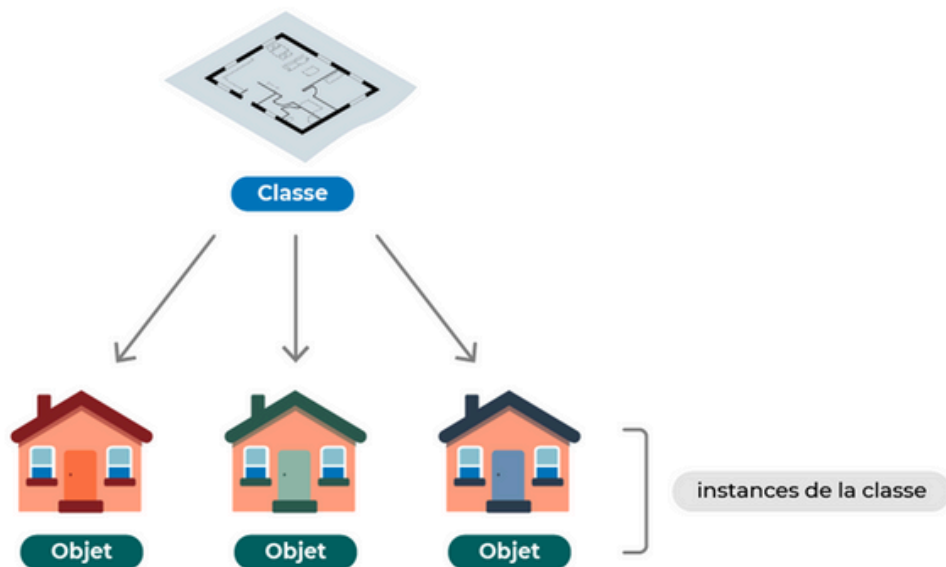


Diagramme de classe :

Conception orientée objet : Représentation du système comme un ensemble d'objets interagissant
Diagramme de classes :

- Représentation de la structure interne du logiciel
- Utilisé surtout en conception mais peut être utilisé en analyse

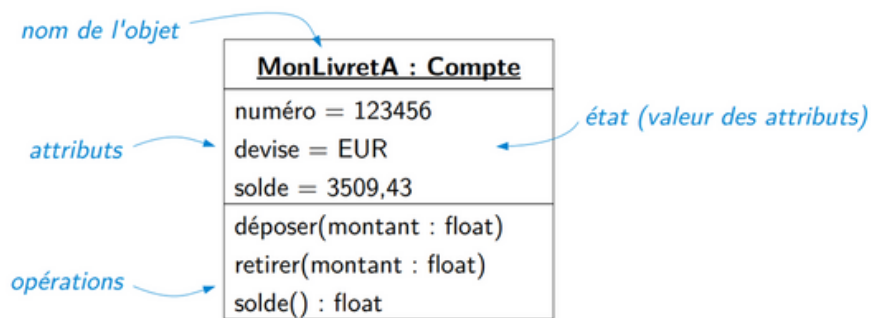
Diagramme d'objets :

- Représentation de l'état du logiciel (objets + relations)
- Diagramme évoluant avec l'exécution du logiciel - création et suppression

d'objets - modification de l'état des objets (valeurs des attributs) - modification des relations entre objets

Objet :

Entité concrète ou abstraite du domaine d'application Décrit par : identité (adresse mémoire) + état (attributs) + comportement (opérations)



Classe : Regroupement d'objets de même nature (mêmes attributs + mêmes opérations) Objet = instance d'une classe

Nom du cours:

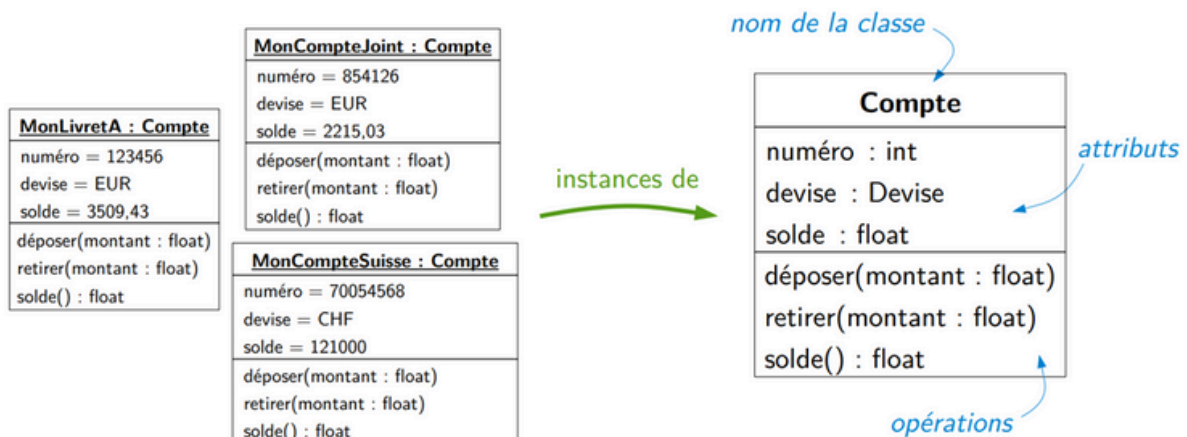
- Le nom de la classe apparaît dans la première partition.

Attributs de classe :

- Les attributs sont affichés dans la deuxième partition.
- Le type d'attribut est affiché après les deux-points.
- Les attributs sont mappés sur des variables membres (membres de données) dans le code.

Opérations de classe (méthodes) :

- Les opérations sont présentées dans la troisième partition. Ce sont des services que la classe fournit.
- Le type de retour d'une méthode est affiché après les deux-points à la fin de la signature de la méthode.
- Le type de retour des paramètres de méthode est affiché après les deux-points suivant le nom du paramètre. Mappage des opérations sur les méthodes de classe dans le code



Encapsulation : La signification de l'encapsulation est de s'assurer que les données « sensibles » sont cachées aux utilisateurs. Pour ce faire, vous devez déclarer les variables/attributs de la classe comme étant privés (non accessibles depuis l'extérieur de la classe). Si vous souhaitez que d'autres personnes puissent lire ou modifier la valeur d'un membre privé, vous pouvez fournir des méthodes publiques get et set.

Exemple :

```
#include <iostream>
using namespace std;

class Employee {
private:
    // Private attribute
    int salary;

public:
    // Setter
    void setSalary(int s) {
        salary = s;
    }
    // Getter
    int getSalary() {
        return salary;
    }
};

int main() {
    Employee myObj;
    myObj.setSalary(50000);
    cout << myObj.getSalary();
    return 0;
}
```

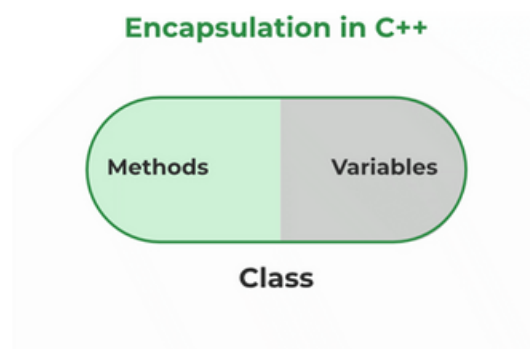
Exemple expliqué :

L'attribut salary est un attribut privé, dont l'accès est restreint.

La méthode publique setSalary() prend un paramètre (s) et l'affecte à l'attribut salary (salary = s).

La méthode publique getSalary() renvoie la valeur de l'attribut private salary.

Dans main(), nous créons un objet de la classe Employee. Nous pouvons maintenant utiliser la méthode setSalary() pour fixer la valeur de l'attribut privé à 50000. Nous appelons ensuite la méthode getSalary() sur l'objet pour renvoyer la valeur.



Passage par valeur: Le passage par valeur est une méthode où une copie de la valeur de l'argument est passée à la fonction. Cela signifie que la fonction travaille avec cette copie et non avec la variable d'origine.

Fonctionnement :

- Lorsqu'une fonction est appelée, une copie des valeurs des arguments est créée dans la mémoire locale de la fonction.
- Toute modification effectuée sur ces copies à l'intérieur de la fonction n'affecte pas la variable originale à l'extérieur de la fonction.

Avantages :

- 1.Sécurité : Les données originales ne sont pas modifiées, ce qui évite les effets secondaires inattendus. Cela permet de travailler de manière plus sûre avec les variables.
- 2.Isolation : Comme la fonction opère sur des copies locales, il y a moins de risque de conflits ou de bugs liés à la modification des variables globales ou externes.

Inconvénients :

- 1.Performance : Si les données sont volumineuses (comme de grandes structures ou des tableaux), la création d'une copie de celles-ci peut entraîner une surcharge mémoire et une baisse des performances.
- 2.Manque de flexibilité : Il est impossible de modifier directement les valeurs originales à partir de la fonction, ce qui peut être limitant dans certains cas où la fonction doit réellement modifier les données d'origine

```
void maMethode(const Object param)
```

Passage par référence : Le passage par référence implique que la fonction reçoit une référence (ou un pointeur) vers la variable originale plutôt qu'une copie de sa valeur. Autrement dit, la fonction travaille directement avec l'adresse mémoire de la variable.

Fonctionnement :

- Lorsqu'une fonction est appelée, elle reçoit un accès direct à la mémoire où la variable originale est stockée.
- Toute modification effectuée sur l'argument dans la fonction affecte directement la variable d'origine.

Avantages :

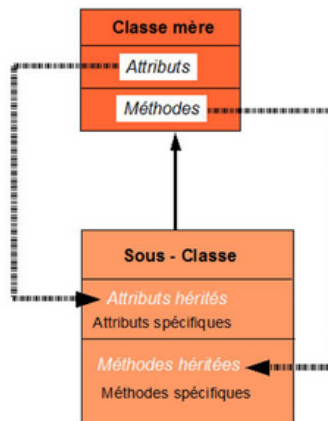
- 1.Performance : Aucune copie des données n'est créée, ce qui est particulièrement avantageux pour les objets volumineux ou complexes. Cela permet de gagner en mémoire et en vitesse.
- 2.Modifications directes : La fonction peut directement modifier les données d'origine, ce qui est utile si la fonction doit apporter des changements à ces données.

Inconvénients :

- 1.Effets secondaires : Les données d'origine peuvent être modifiées de manière non anticipée, ce qui peut entraîner des bugs difficiles à identifier, surtout si la fonction est complexe.
- 2.Moins de sécurité : Comme la fonction travaille directement avec les données originales, toute modification non voulue ou erreur de programmation peut affecter l'état de l'application.
- 3.Complexité : La gestion des références peut être plus complexe, notamment lorsqu'il y a plusieurs références à une même donnée, ce qui peut rendre le suivi des changements plus difficile.

```
void maMethode(const Object &param)  
void maMethode(const Object *param)
```

Héritage: L'héritage est un mécanisme qui a été introduit dans la programmation objet dans le but d'éviter la réécriture inutile de code lorsqu'une classe n'est qu'un cas spécial d'une autre classe:



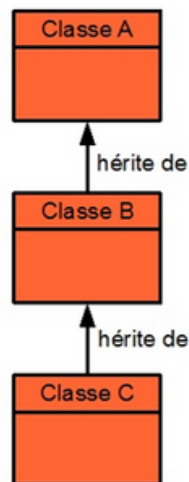
La classe plus spécialisée est appelée sous-classe (ou classe fille) et la classe plus générale, classe mère.

Lorsqu'une relation d'héritage est définie entre deux classes, la sous-classe hérite de tous les attributs et méthodes de la classe mère. C'est à dire que tout se passe comme si les attributs et méthodes de la classe mère avaient été explicitement définies pour la sous-classe.

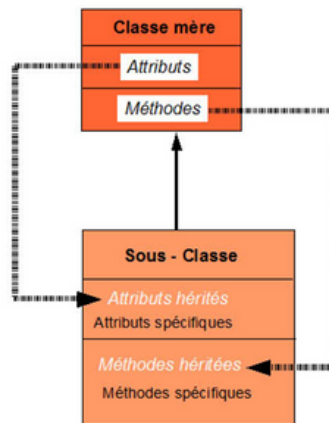
Une sous-classe d'une classe donnée représente donc une sorte de cas particulier de cette classe qui possède des attributs méthodes supplémentaires et spécifiques.

Notez que la relation d'héritage entre classes peut se faire à plusieurs niveaux: la classe mère d'une classe, peut également avoir une classe mère ... etc.

Prenons le cas d'un héritage à trois niveaux: la classe C hérite de la classe B, qui elle-même hérite de la classe A:



Héritage: L'héritage est un mécanisme qui a été introduit dans la programmation objet dans le but d'éviter la réécriture inutile de code lorsqu'une classe n'est qu'un cas spécial d'une autre classe:



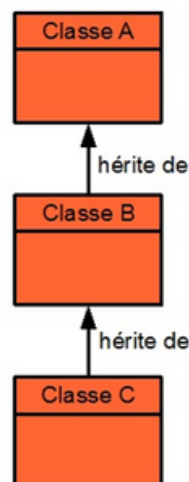
La classe plus spécialisée est appelée sous-classe (ou classe fille) et la classe plus générale, classe mère.

Lorsqu'une relation d'héritage est définie entre deux classes, la sous-classe hérite de tous les attributs et méthodes de la classe mère. C'est à dire que tout se passe comme si les attributs et méthodes de la classe mère avaient été explicitement définies pour la sous-classe.

Une sous-classe d'une classe donnée représente donc une sorte de cas particulier de cette classe qui possède des attributs méthodes supplémentaires et spécifiques.

Notez que la relation d'héritage entre classes peut se faire à plusieurs niveaux: la classe mère d'une classe, peut également avoir une classe mère ... etc.

Prenons le cas d'un héritage à trois niveaux: la classe C hérite de la classe B, qui elle-même hérite de la classe A:

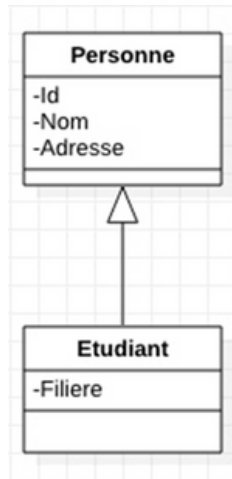


Dans ce cas, la classe C héritera de tous les attributs et méthodes de la classe B, mais également de ceux de la classe A. A et B sont des classes ascendantes de la classe C et inversement B et C sont des classes descendantes de la classe A.

De manière générale, une classe hérite donc des attributs et méthodes de toutes ses classes ascendantes.

Héritage en C++ : En C++, il existe plusieurs manières de déclarer qu'une classe B est une sous-classe d'une classe A : En la déclarant de cette manière, la classe B hérite de toutes les membres publiques de la classe A. Attention : les membres privés ne sont pas hérités.

Héritage simple : Dans ce type d'héritage, une classe dérivée hérite d'une seule classe de base. C'est la forme la plus simple de l'héritage.



```
class Personne
{
    private:
        int Id;
        char Nom[30];
        char Adresse[100];

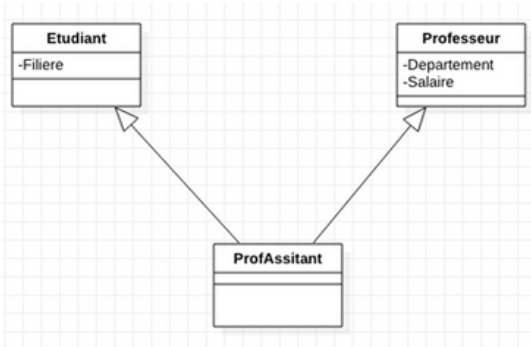
    public:
        Personne(int, char[], char[]);
        void afficher();
};

class Etudiant : public Personne
{
    private:
        char Filiere[40];

    public:
        Etudiant(int, char[], char[], char[]);
};
```

Héritage multiple :

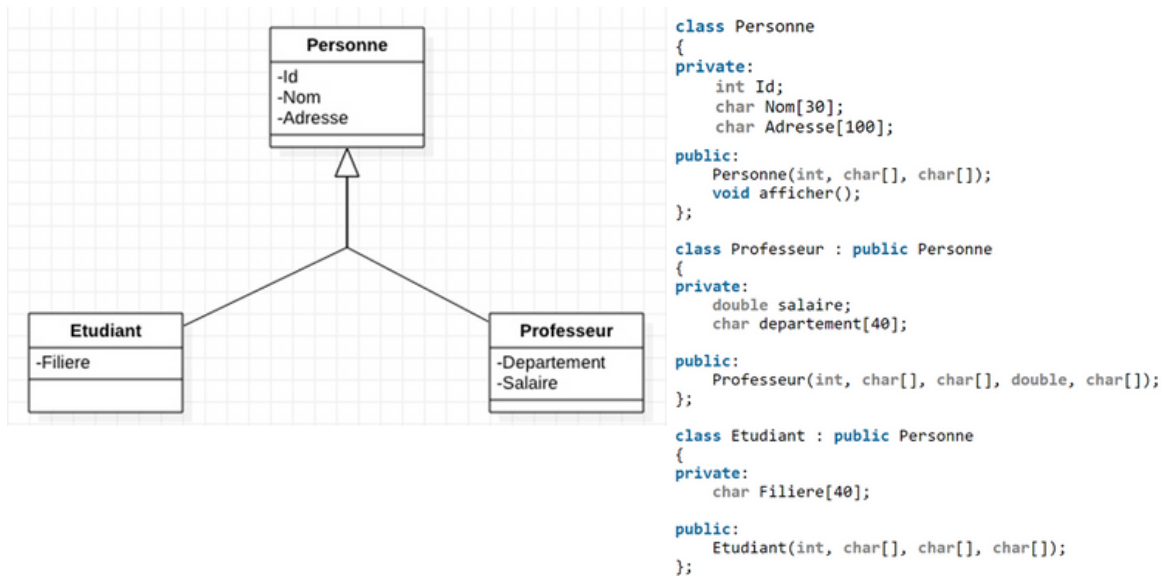
Dans ce type d'héritage, une seule classe dérivée peut hériter de deux ou plus de deux classes de base.



```
class ProfAssistant : public Etudiant, public Professeur
{
};
```

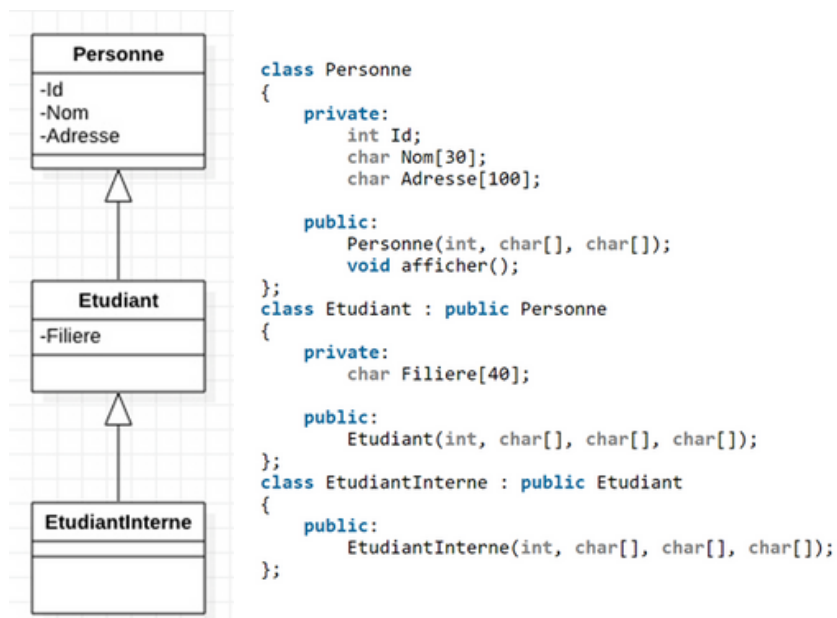
Héritage hiérarchique :

Dans ce type d'héritage, une classe de base a plus d'une classe fille. Par exemple:



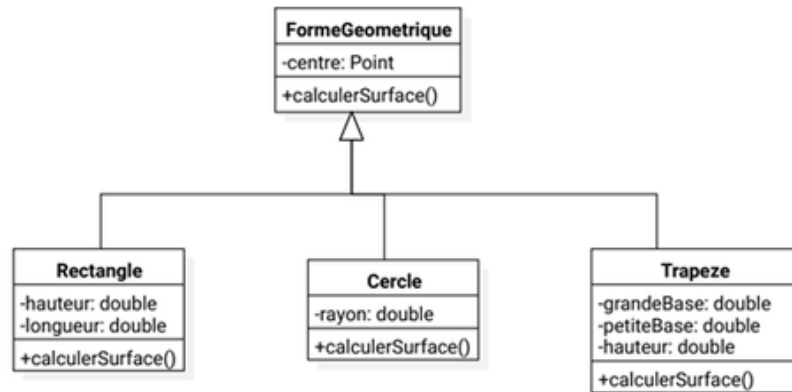
Héritage à plusieurs niveaux :

Dans ce type d'héritage, la classe dérivée hérite d'une classe, qui hérite à son tour d'une autre classe. La classe de base pour l'une est la sous-classe pour l'autre.



Notion de polymorphisme :

Parfois certaines méthodes de la classe mère nécessitent une certaine adaptation, il est donc possible de les redéfinir :



Ici, lorsque l'on invoque la méthode `calculerSurface()` sur un objet de type `Cercle`, c'est la méthode de `Cercle` qui sera invoquée (et pas celle de la classe mère).

Poly = plusieurs, morphisme = forme :

Définition - Polymorphisme

- Propriété d'un élément de pouvoir se présenter sous plusieurs formes
- Capacité donnée à une même opération de s'effectuer différemment suivant le contexte de la classe où elle se trouve

Dans le cas des figures géométriques, la fonction `calculerSurface()` est une fonction polymorphe. Le polymorphisme est réalisable grâce à la Liaison Dynamique :

Liaison dynamique

Mécanisme permettant que la définition d'une méthode soit décidée au moment de l'exécution de celle-ci (et non à la compilation).

le polymorphisme ad-hoc via le mécanisme de surcharge :

En programmation orientée objet, la surcharge, aussi appelée « overloading », consiste à déclarer, dans une même classe, deux méthodes de même nom mais avec des sémantiques différentes :

- Même nom de méthode,
- Paramètres différents (soit sur le nombre ou le/les type(s)),
- Le type de retour n'est pas pris en compte.

```
class A{
    public static void F(){}
    public static int F(int i){return i;}
    public static double F(double i, double j){return i;}
    public static double F(int i, double j){return j;}
    public static double F(double i, int j){return i;}
    public static void F(string b){}
    public static void F(string b, string a){}
}
```

Polymorphisme d'héritage:

Le polymorphisme d'héritage va consister à redéclarer dans une classe fille, une méthode déjà déclarée dans une classe parent.

Contrairement au polymorphisme ad hoc qui n'utilise pas de mot clé particulier, on utilisera ici le couple virtual/override. La première déclaration de la méthode doit porter le qualificatif virtual et toutes les classes enfants modifiant son comportement doivent précéder sa déclaration par le qualificatif override.

```
class A {
    public virtual void Afficher(){
        WriteLine("Bonjour");
    }
}
class B : A
{ }
class C : B {
    public override void Afficher() {
        WriteLine("Coucou");
    }
}
A obj1 = new A();
C obj2 = new C();
obj1.Afficher();           // résultat => Bonjour
obj2.Afficher();           // résultat => Coucou
```


Quand on utilise une méthode qualifiée avec `virtual/override`, le CLR doit donc décider, à chaque fois que la méthode est appelée, quelle version doit être exécutée selon le type constaté de l'objet. Cette décision ne peut pas être prise au moment de la compilation du programme (seul le type déclaré est connu à ce moment) et a donc un coût lors de l'exécution du programme. Ce coût est généralement négligeable mais peut devenir important lorsque la méthode est appelée très fréquemment. Pour en savoir plus, vous pouvez regarder comment sont généralement implémentées les méthodes virtuelles avec les vtables .

⚠ Attention
Si l'on oublie les mots clés `virtual/override`, cela ne provoquera pas d'erreur (ni à la compilation, ni à l'exécution) mais le comportement ne sera pas celui souhaité:

```
class A {
    public void Afficher(){
        WriteLine("Bonjour");
    }
}

class B : A
{
}

class C : B {
    public void Afficher() {
        WriteLine("Coucou");
    }
}

class D : C
{
    public void Afficher(){
        WriteLine("Hello");
    }
}

A[] Liste = new A[4];
Liste[0] = new A();
Liste[1] = new B();
Liste[2] = new C();
Liste[3] = new D();

foreach(A m in Liste)
    m.Afficher(); // Résultats : Bonjour Bonjour Bonjour Bonjour
```

Cela pose problème car les objets sont gérés comme s'ils étaient **TOUS** de type **A** ; c'est uniquement le type déclaré qui est pris en compte. En l'absence de toute syntaxe particulière, le langage considère que le type de l'objet référencé est donné par le type de la référence. On parle de **masquage**.

Différences clés :

1. Polymorphisme d'héritage :

- Résolu à l'exécution (liaison tardive)
- Utilise des méthodes virtuelles
- Permet la substitution (un `CLpoint3D` peut être utilisé partout où un `CLpoint` est attendu)

2. Polymorphisme Ad-Hoc :

- Résolu à la compilation (liaison précoce)
- Utilise la surcharge de méthodes
- Le choix de la méthode dépend du type statique des arguments

Template : Les templates en C++ sont un outil puissant qui permet d'écrire du code générique. Ils permettent de créer des fonctions et des classes qui peuvent fonctionner avec n'importe quel type de données.

Templates de Fonction :

Les templates de fonction permettent de créer des fonctions génériques. Par exemple, une fonction pour trouver le maximum de deux valeurs peut être écrite comme suit :

```
template <typename T>
T max(T a, T b) {
    return (a > b) ? a : b;
}
```

Où mettre la fonction ?

Habituellement, un programme est subdivisé en plusieurs fichiers que l'on classe en deux catégories :

1. Les fichiers de code (les .cpp).
2. Les fichiers d'en-tête (les .hpp).

Généralement, on met le prototype de la fonction dans un .hpp et la définition dans le .cpp .



Pour les fonctions templates, c'est différent.

TOUT doit obligatoirement se trouver dans le fichier .hpp , sinon votre programme ne pourra pas compiler.

Templates de Classe

Les templates de classe permettent de créer des classes génériques. Par exemple, une classe de pile (stack) générique peut être écrite comme suit :

```
template <typename T>
class Stack {
private:
    std::vector<T> elements;
public:
    void push(T const& elem) {
        elements.push_back(elem);
    }
    void pop() {
        if (!elements.empty()) {
            elements.pop_back();
        }
    }
    T top() const {
        return elements.back();
    }
    bool empty() const {
        return elements.empty();
    }
};
```

Définition dans et hors-classe (in-class et out-of-line) en C++: En C++, les définitions de fonctions membres peuvent être faites soit à l'intérieur de la classe (in-class), soit à l'extérieur de la classe (out-of-line). Voici une explication de chaque méthode :

Définition In-Class

La définition in-class signifie que la fonction membre est définie directement dans la déclaration de la classe. Cela peut rendre le code plus lisible et plus facile à comprendre, surtout pour les petites fonctions. Par exemple :

```
class MyClass {
public:
    void display() {
        std::cout << "Hello, World!" << std::endl;
    }
};
```

Définition Out-of-Line

La définition out-of-line signifie que la fonction membre est définie en dehors de la déclaration de la classe. Cela peut aider à garder la déclaration de la classe propre et concise, surtout pour les fonctions plus longues. Pour définir une fonction out-of-line, vous devez utiliser l'opérateur de résolution de portée ::.

Par exemple :

```
class MyClass {
public:
    void display();
};

// Définition de la fonction en dehors de la classe
void MyClass::display() {
    std::cout << "Hello, World!" << std::endl;
}
```

Avantages et Inconvénients

In-Class

- Avantages :
 - Plus lisible pour les petites fonctions.
 - Peut améliorer les performances grâce à l'optimisation du compilateur (inline).
- Inconvénients :
 - Peut rendre la déclaration de la classe encombrée si les fonctions sont longues.

Out-of-Line

- Avantages :
 - Garde la déclaration de la classe propre et concise.
 - Sépare l'interface de l'implémentation, ce qui peut améliorer la lisibilité du code.
- Inconvénients :
 - Peut être moins lisible pour les petites fonctions.
 - Nécessite l'utilisation de l'opérateur de résolution de portée ::.

la notion de design pattern (observateur et observable). Les design patterns "Observateur" et "Observable" sont des modèles de conception comportementaux qui permettent de définir une relation de dépendance entre objets, où un objet (l'observateur) est informé des changements d'état d'un autre objet (l'observable). Voici une explication détaillée de ces concepts :

Pattern Observateur (Observer)

Le pattern Observateur permet à un objet (l'observateur) de s'abonner à un autre objet (l'observable) pour être notifié des changements d'état. Ce modèle est souvent utilisé dans les interfaces graphiques et les systèmes de notification.

Pattern Observable (Subject)

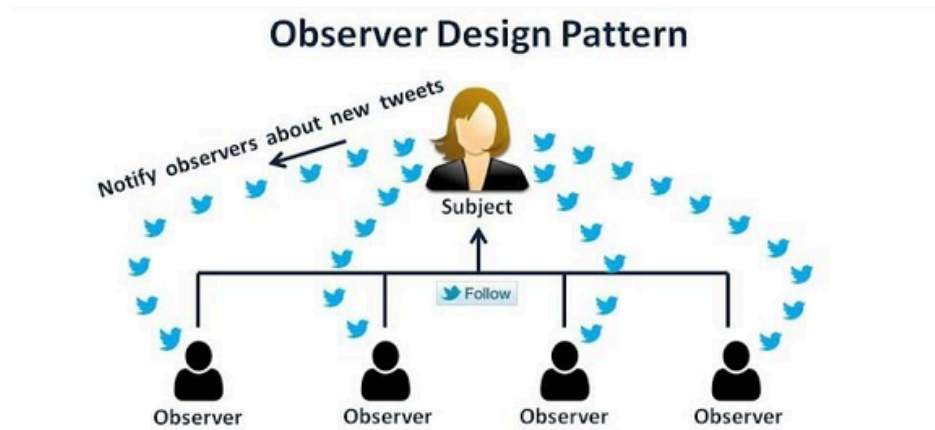
Le pattern Observable est souvent utilisé en conjonction avec le pattern Observateur. L'objet observable maintient une liste d'observateurs et les notifie des changements d'état.

Avantages

- **Découplage** : Les observateurs et les observables sont faiblement couplés, ce qui améliore la modularité et la flexibilité du code.
- **Réutilisabilité** : Les observateurs peuvent être réutilisés avec différents observables.

Inconvénients

- **Complexité** : Peut ajouter de la complexité au code, surtout avec de nombreux observateurs.
- **Performance** : La notification de nombreux observateurs peut affecter les performances.



Architecture Logicielle (3 couches)

L'architecture logicielle à trois couches est un modèle de conception couramment utilisé pour structurer les applications logicielles. Elle divise l'application en trois couches distinctes, chacune ayant des responsabilités spécifiques. Voici une explication détaillée :

1. Couche de Présentation (UI)

- Rôle : Cette couche est responsable de l'interface utilisateur. Elle gère l'affichage des données et l'interaction avec l'utilisateur.
- Exemples : Pages web, applications mobiles, interfaces graphiques.
- Technologies : HTML, CSS, JavaScript, frameworks comme React, Angular, etc.

2. Couche Métier (Logique)

- Rôle : Cette couche contient la logique métier de l'application. Elle traite les données, applique les règles métier et effectue les calculs nécessaires.
- Exemples : Calculs, validations, traitement des données.
- Technologies : Langages de programmation comme Java, C#, Python, frameworks comme Spring, .NET, etc.

3. Couche de Données (Accès aux Données)

- Rôle : Cette couche est responsable de la gestion des données. Elle interagit avec les bases de données pour stocker, récupérer et manipuler les données.
- Exemples : Requêtes SQL, accès aux bases de données, gestion des transactions.
- Technologies : SGBD comme MySQL, PostgreSQL, MongoDB, ORM comme Hibernate, Entity Framework, etc.

Avantages de l'architecture à trois couches :

- Modularité : Chaque couche est indépendante, ce qui facilite la maintenance et les mises à jour.
- Réutilisabilité : Les composants de chaque couche peuvent être réutilisés dans d'autres applications.
- Scalabilité : L'application peut être facilement mise à l'échelle en modifiant ou en ajoutant des composants dans une couche spécifique.